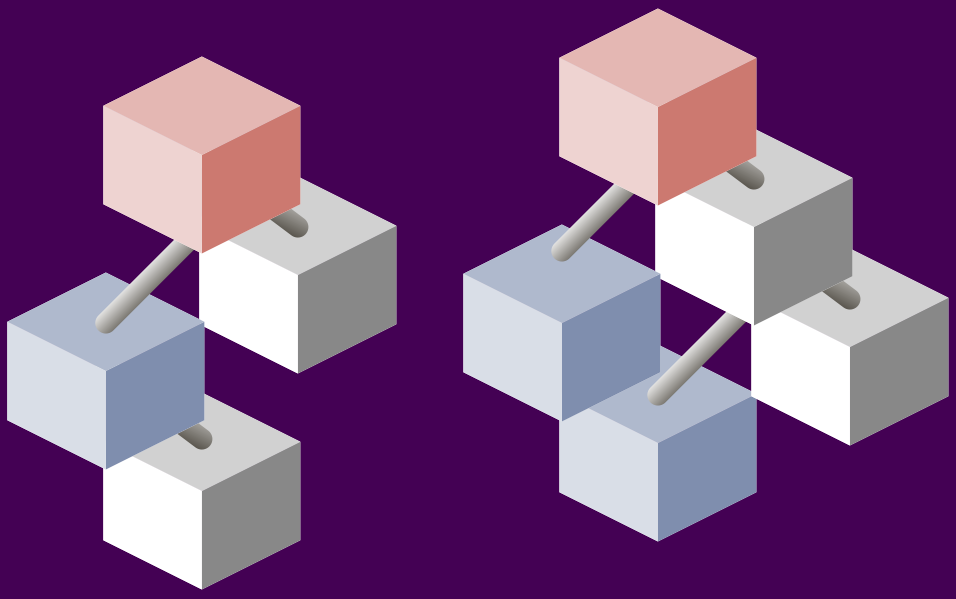
Treading on Python Series



EFFECTIVE XGBOOST

Tuning, Understanding, and Deploying Classification Models

Matt Harrison

Effective XGBoost

Tuning, Understanding, and Deploying Classification Models

Effective XGBoost

Tuning, Understanding, and Deploying Classification Models

Matt Harrison

Technical Editors: Edward Krueger, Alex Rook, Ronald Legere

hairysun.com

COPYRIGHT © 2023

While every precaution has been taken in the preparation of this book, the publisher and author assumes no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

Contents

1 Introduction **3**

2 Datasets **5**
2.1 Cleanup . . . 5
2.2 Cleanup Pipeline . . . 6
2.3 Summary . . . 13
2.4 Exercises . . . 14

3 Exploratory Data Analysis **15**
3.1 Correlations . . . 15
3.2 Bar Plot . . . 17
3.3 Summary . . . 21
3.4 Exercises . . . 21

4 Tree Creation **23**
4.1 The Gini Coefficient . . . 23
4.2 Coefficients in Trees . . . 27
4.3 Another Visualization Tool . . . 30
4.4 Summary . . . 30
4.5 Exercises . . . 30

5 Stumps on Real Data **31**
5.1 Scikit-learn stump on real data . . . 31
5.2 Decision Stump with XGBoost . . . 33
5.3 Values in the XGBoost Tree . . . 35
5.4 Summary . . . 37
5.5 Exercises . . . 37

6 Model Complexity & Hyperparameters **39**
6.1 Underfit . . . 39
6.2 Growing a Tree . . . 39
6.3 Overfitting . . . 40
6.4 Overfitting with Decision Trees . . . 40
6.5 Summary . . . 41
6.6 Exercises . . . 42

7 Tree Hyperparameters **43**
7.1 Decision Tree Hyperparameters . . . 43
7.2 Tracking changes with Validation Curves . . . 44
7.3 Leveraging Yellowbrick . . . 45
7.4 Grid Search . . . 47

7.5 Summary . . . 49
7.6 Exercises . . . 49

8 Random Forest **51**
8.1 Ensembles with Bagging . . . 51
8.2 Scikit-learn Random Forest . . . 51
8.3 XGBoost Random Forest . . . 53
8.4 Random Forest Hyperparameters . . . 54
8.5 Training the Number of Trees in the Forest . . . 57
8.6 Summary . . . 58
8.7 Exercises . . . 58

9 XGBoost **59**
9.1 Jargon . . . 59
9.2 Benefits of Boosting . . . 59
9.3 A Big Downside . . . 60
9.4 Creating an XGBoost Model . . . 60
9.5 A Boosted Model . . . 61
9.6 Understanding the Output of the Trees . . . 62
9.7 Summary . . . 65
9.8 Exercises . . . 65

10 Early Stopping **67**
10.1 Early Stopping Rounds . . . 67
10.2 Plotting Tree Performance . . . 68
10.3 Different `eval_metrics` . . . 71
10.4 Summary . . . 73
10.5 Exercises . . . 73

11 XGBoost Hyperparameters **75**
11.1 Hyperparameters . . . 75
11.2 Examining Hyperparameters . . . 77
11.3 Tuning Hyperparameters . . . 78
11.4 Intuitive Understanding of Learning Rate . . . 78
11.5 Grid Search . . . 80
11.6 Summary . . . 83
11.7 Exercises . . . 83

12 Hyperopt **85**
12.1 Bayesian Optimization . . . 85
12.2 Exhaustive Tuning with Hyperopt . . . 85
12.3 Defining Parameter Distributions . . . 88
12.4 Exploring the Trials . . . 92
12.5 EDA with Plotly . . . 100
12.6 Conclusion . . . 103
12.7 Exercises . . . 104

13 Step-wise Tuning with Hyperopt **105**
13.1 Groups of Hyperparameters . . . 105
13.2 Visualization Hyperparameter Scores . . . 106
13.3 Training an Optimized Model . . . 106
13.4 Summary . . . 108
13.5 Exercises . . . 108

14 Do you have enough data? 109
14.1 Learning Curves . . . 109
14.2 Learning Curves for Decision Trees . . . 110
14.3 Underfit Learning Curves . . . 111
14.4 Overfit Learning Curves . . . 111
14.5 Summary . . . 112
14.6 Exercises . . . 112

15 Model Evaluation 115
15.1 Accuracy . . . 115
15.2 Confusion Matrix . . . 115
15.3 Precision and Recall . . . 117
15.4 F1 Score . . . 120
15.5 ROC Curve . . . 120
15.6 Threshold Metrics . . . 123
15.7 Cumulative Gains Curve . . . 125
15.8 Lift Curves . . . 126
15.9 Summary . . . 127
15.10Exercises . . . 127

16 Training For Different Metrics 129
16.1 Metric overview . . . 129
16.2 Training with Validation Curves . . . 129
16.3 Step-wise Recall Tuning . . . 130
16.4 Summary . . . 132
16.5 Exercises . . . 132

17 Model Interpretation 133
17.1 Logistic Regression Interpretation . . . 133
17.2 Decision Tree Interpretation . . . 134
17.3 XGBoost Feature Importance . . . 137
17.4 Surrogate Models . . . 138
17.5 Summary . . . 140
17.6 Exercises . . . 140

18 xgbfir (Feature Interactions Reshaped) 141
18.1 Feature Interactions . . . 141
18.2 xgbfir . . . 141
18.3 Deeper Interactions . . . 147
18.4 Specifying Feature Interactions . . . 148
18.5 Summary . . . 148
18.6 Exercises . . . 149

19 Exploring SHAP 151
19.1 SHAP . . . 151
19.2 Examining a Single Prediction . . . 155
19.3 Waterfall Plots . . . 156
19.4 A Force Plot . . . 160
19.5 Force Plot with Multiple Predictions . . . 160
19.6 Understanding Features with Dependence Plots . . . 160
19.7 Jittering a Dependence Plot . . . 162
19.8 Heatmaps and Correlations . . . 162

19.9 Beeswarm Plots of Global Behavior . . . 165
19.10 SHAP with No Interaction . . . 166
19.11 Summary . . . 169
19.12 Exercises . . . 169

20 Better Models with ICE, Partial Dependence, Monotonic Constraints, and Calibration 171
20.1 ICE Plots . . . 171
20.2 ICE Plots with SHAP . . . 176
20.3 Partial Dependence Plots . . . 176
20.4 PDP with SHAP . . . 179
20.5 Monotonic Constraints . . . 180
20.6 Calibrating a Model . . . 185
20.7 Calibration Curves . . . 185
20.8 Summary . . . 186
20.9 Exercises . . . 187

21 Serving Models with MLFlow 189
21.1 Installation and Setup . . . 189
21.2 Inspecting Model Artifacts . . . 192
21.3 Running A Model From Code . . . 193
21.4 Serving Predictions . . . 193
21.5 Querying from the Command Line . . . 193
21.6 Querying with the Requests Library . . . 198
21.7 Building with Docker . . . 198
21.8 Conclusion . . . 199
21.9 Exercises . . . 199

22 Conclusion 201
22.1 One more thing . . . 201

Index 205

Forward

"XGBoost is all you need."It started as a half-joke, a silly little phrase that I myself had not been too confident about, but it grew into a signature line of sorts. I don't remember when I first said it, and I can only surmise what motivated me to say it. Part of it was my frustration with the increasingly complex deep learning that was being pushed as the only valid approach to tackling all Machine Learning problems. The other part of it came from a sense of maturity that was hard won over the years, acquired from many different wild experiments with various ML tools and techniques, particularly those that deal with tabular data problems. The more problems you tackle, the more you appreciate such things as parsimony, robustness, and elegance in implementing your machine learning models and pipelines.

My first exposure to XGBoost came on Kaggle. Over the past decade, the progress of Machine Learning for tabular data has been intertwined with Machine Learning platforms, and Kaggle in particular. XGBoost was first announced on Kaggle in 2015, not long before I joined that platform. XGBoost was initially a research project by Tianqi Chen, but due to its excellent predictive performance on a variety of machine learning tasks it quickly became the algorithm of choice for many Data Science practitioners.

XGBoost is both a library and a particular gradient boosted trees (GBT) algorithm. (Although the XGBoost library also supports other - linear - base learners.) GBTs are a class of algorithms that utilize the so-called ensembling - building a very strong ML algorithm by combining many weaker algorithms. GBTs use decision trees as "base learners", utilize "boosting"as an ensembling method, and optimize the ensembling by utilizing gradient descent, something that they have in common with other machine learning methods, such as neural networks.

XGBoost the library is in its own right one of the main reasons why I am such a big XGBoost promoter. As one of the oldest and most widely used GBT libraries, it has matured and become very robust and stable. It can be easily installed and used on almost any computing environment. I have installed it and used it on everything from Raspberry Pi Zero to DGX Station A100. It has had a stable version for various Arm-based platforms for a long while, including the new Macs with Apple chips. It can even run natively in the browser - it has been included in Wasm and PyScript for a while. XGBoost is the only GBT library with a comprehensive Nvidia CUDA GPU support. It works on a single machine, or on a large cluster. And since version 1.7, it also supports federated learning. It includes C, Java, Python, and R front ends, as well as many other ones. If you are a working Data Scientist, and need an efficient way to train and deploy a Machine Learning model for a wide variety of problems, chances are that XGBoost is indeed all you need.

To be clear, in terms of predictive performance, I have always considered XGBoost to be on par with other well known GBT libraries - LightGBM, CatBoost, HistGradientBoosting, etc. Each one of them has its strengths and weaknesses, but in the first approximation all of them are fairly interchangeable with each other for the majority of problems that a working Data Scientist comes across. In my experience there is no a priori way of telling which one of them will perform the best on any given dataset, but in terms of practical considerations the

differences are usually negligible. Mastering any one of them will go a long way towards a user being comfortable with all of them.

Even though GBTs are very widely used and the gold standard for tackling tabular data Machine Learning problems, there is still a serious paucity of learning material for them. Most of us have learned to use them by following widely scattered blog posts, Kaggle competitions, GitHub repos, and many other dispersed pieces of information. There are a couple good introductory books out there as well, but for the most part one has to search wide to find a very comprehensive instruction manual that can get you up and running with XGBoost. This book aims to put a lot of that scattered material into one coherent whole, with a straightforward and logical content progression.

I've now known Matt for years, and I greatly admire his pedagogical perspective. He is a non-nonsense educator, and has a very down to earth approach to all of his teaching. I've greatly benefited from several of his books, especially the ML handbook and the Effective Pandas book. In all of his books he aims to get the reader to the practical parts as soon as possible, and this book is no exception - you start dealing with code from the very first page. Like with all of his other books, this one was also written with a working practitioner in mind. The book skips most of the convoluted theoretical background, and delves directly into practical examples. You should be comfortable coding and ideally be an advanced beginner to an intermediate Python coder in order to get the most out of this book.

Machine Learning for tabular data is still a very hands-on artisanal process. A big part of what makes a great tabular data ML model has to do with proper data preparation and feature engineering. This is where Matt's background with Pandas really comes in handy - many Pandas examples throughout the book are exceptionally valuable in their own right. Chapters end with a great selection of useful exercises. All of the code in the book is also available from the accompanying repo, and most of the datasets can be found on Kaggle.

Like all other great tools, it takes many years of experience and dedicated work to fully master XGBoost. Every path to full mastery needs to start somewhere, and I can't think of a better starting point than this book. If you get through all of it you will be well prepared for the rest of your Machine Learning for tabular data journey.

Happy Boosting!

Bojan Tunguz

Chapter 1

Introduction

In this book, we will build our intuition for classification with supervised learning models. Initially, we will look at decision trees, a fundamental component of the XGBoost model, and consider the tradeoffs we make when creating a predictive model. If you have that background, feel free to skip these chapters and jump straight to the XGBoost chapters.

While providing best practices for using the XGBoost library, we will also show many related libraries and how to use them to improve your model. Finally, we will demonstrate how to deploy your model.

I strongly encourage you to practice using this library on a dataset of your choice. Each chapter has suggested exercises for you to consider what you have learned and put it into practice. Practicing writing code is crucial for learning because it helps to solidify understanding of the library and understanding concepts. Additionally, regular coding practice can build familiarity with coding syntax and increase coding efficiency over time. Using your fingers to practice will help you much more than just reading.

If you are interested in courses to practice and learn more Python and data materials, check out https://store.metasnake.com. Use the code XGBOOK for a discount.

Chapter 2

Datasets

We will be using machine learning classifiers to predict labels for data. I will demonstrate the features of XGBoost using survey data from Kaggle. Kaggle is a company that hosts machine learning competitions. They have conducted surveys with users about their backgrounds, experience, and tooling. Our goal will be to determine whether a respondent's job title is "Data Scientist" or "Software Engineer" based on how they responded to the survey.

This data comes from a Kaggle survey conducted in 2018 [1]. As part of the survey, Kaggle asked users about a wide range of topics, from what kind of machine learning software they use to how much money they make. We're going to be using the responses from the survey to predict what kind of job the respondent has.

2.1 Cleanup

I've hosted the data on my GitHub page. The 2018 survey data is in a ZIP file. Let's use some Python code to extract the contents of this data into a Pandas DataFrame. If you need help setting up your environment, see the appendix.

The ZIP file contains multiple files. We are concerned with the *multipleChoiceResponses.csv* file.

```
import pandas as pd

import urllib.request
import zipfile


url = 'https://github.com/mattharrison/datasets/raw/master/data/'\
    'kaggle-survey-2018.zip'
fname = 'kaggle-survey-2018.zip'
member_name = 'multipleChoiceResponses.csv'


def extract_zip(src, dst, member_name):
    """Extract a member file from a zip file and read it into a pandas
    DataFrame.

    Parameters:
        src (str): URL of the zip file to be downloaded and extracted.
```

[1] https://www.kaggle.com/datasets/kaggle/kaggle-survey-2018

```
        dst (str): Local file path where the zip file will be written.
        member_name (str): Name of the member file inside the zip file
            to be read into a DataFrame.

    Returns:
        pandas.DataFrame: DataFrame containing the contents of the
            member file.
    """
    url = src
    fname = dst
    fin = urllib.request.urlopen(url)
    data = fin.read()
    with open(dst, mode='wb') as fout:
        fout.write(data)
    with zipfile.ZipFile(dst) as z:
        kag = pd.read_csv(z.open(member_name))
        kag_questions = kag.iloc[0]
        raw = kag.iloc[1:]
        return raw

raw = extract_zip(url, fname, member_name)
```

I created the function `extract_zip` so I can reuse this functionality. I will also add this function to a library that I will develop as we progress, `xg_helpers.py`.

After running the above code, we have a Pandas dataframe assigned to the variable `raw`. This is the raw data of survey responses. We show how to explore it and clean it up for machine learning.

2.2 Cleanup Pipeline

The raw data has over 23,000 rows and almost 400 columns. Most data is not natively found in a form where you can do machine learning on it. Generally, you will need to perform some *preprocessing* on it.

Some of the columns don't lend easily to analysis because they aren't in numeric form. Additionally, there may be missing data that needs to be dealt with. I will show how to preprocess the data using the Pandas library and the Scikit-learn library. I'll describe the steps, but I will not dive deeply into the features of the Pandas library. I suggest you check out *Effective Pandas* for a comprehensive overview of that library.

Our task will be transforming survey responses into numeric values, encoding categorical values into numbers, and filling in missing values. We will use a *pipeline* for that. Scikit-learn pipelines are a convenient tool for tying together the steps of constructing and evaluating machine learning models. A pipeline chains a series of steps, including feature extraction, dimensionality reduction, and model fitting, into a single object. Pipelines simplify the end-to-end process of machine learning and make you more efficient. They also make it easy to reuse and share your work and eliminate the risk of errors by guaranteeing that each step is executed in a precise sequence.

I wrote a function, `tweak_kag`, to perform the survey response cleanup. I generally create a cleanup function every time I get new tabular data to work with.

The `tweak_kag` function is the meat of the Pandas preprocessing logic. It uses a chain to manipulate the data one action (I typically write each step on its own line) at a time. You can read it as a recipe of actions. The first is to use `.assign` to create or update columns. Here are the columns updates:

- *age* - Pull off the first two characters of the *Q2* column and convert them to an integer.
- *education* - Replace the education strings with numeric values.
- *major* - Take the top three majors, change the others to `'other'`, and then rename those top three majors to shortened versions.
- *year_exp* - Convert the *Q8* column to years of experience by replacing `'+'` empty space, splitting on `'-'` (the first value of the range) and taking the left-hand side, and converting that value to a floating point number.
- *compensation* - Replace values in the *Q9* column by removing commas, shortening 500,000 to 500, splitting on `'-'` (the first value of the range) and taking the left side, filling in missing values with zero, and converting that value to an integer and multiplying it by 1,000.
- *python* - Fill in missing values of *Q16_Part_1* with zero and convert the result to an integer.
- *r* - Fill in missing values of *Q16_Part_2* with zero and convert the result to an integer.
- *sql* - Fill in missing values of *Q16_Part_2* with zero and convert the result to an integer.

After column manipulation, `tweak_kag` renames the columns by replacing spaces with an underscore.

Finally, it pulls out only the *Q1*, *Q2*, *age*, *education*, *major*, *years_exp*, *compensation*, *python*, *r*, and *sql* columns.

```
def tweak_kag(df_: pd.DataFrame) -> pd.DataFrame:
    """
    Tweak the Kaggle survey data and return a new DataFrame.

    This function takes a Pandas DataFrame containing Kaggle
    survey data as input and returns a new DataFrame. The
    modifications include extracting and transforming certain
    columns, renaming columns, and selecting a subset of columns.

    Parameters
    ----------
    df_ : pd.DataFrame
        The input DataFrame containing Kaggle survey data.

    Returns
    -------
    pd.DataFrame
        The new DataFrame with the modified and selected columns.
    """
    return (df_
            .assign(age=df_.Q2.str.slice(0,2).astype(int),
                    education=df_.Q4.replace({'Master's degree': 18,
                         'Bachelor's degree': 16,
                         'Doctoral degree': 20,
'Some college/university study without earning a bachelor's degree': 13,
                         'Professional degree': 19,
                         'I prefer not to answer': None,
                         'No formal education past high school': 12}),
                    major=(df_.Q5
                              .pipe(topn, n=3)
                              .replace({
```

```
                        'Computer science (software engineering, etc.)': 'cs',
                        'Engineering (non-computer focused)': 'eng',
                        'Mathematics or statistics': 'stat'})
                         ),
                    years_exp=(df_.Q8.str.replace('+','', regex=False)
                           .str.split('-', expand=True)
                           .iloc[:,0]
                           .astype(float)),
                    compensation=(df_.Q9.str.replace('+','', regex=False)
                           .str.replace(',','', regex=False)
                           .str.replace('500000', '500', regex=False)
  .str.replace('I do not wish to disclose my approximate yearly compensation',
               '0', regex=False)
                           .str.split('-', expand=True)
                           .iloc[:,0]
                           .fillna(0)
                           .astype(int)
                           .mul(1_000)
                                   ),
                    python=df_.Q16_Part_1.fillna(0).replace('Python', 1),
                    r=df_.Q16_Part_2.fillna(0).replace('R', 1),
                    sql=df_.Q16_Part_3.fillna(0).replace('SQL', 1)
                )#assign
            .rename(columns=lambda col:col.replace(' ', '_'))
            .loc[:, 'Q1,Q3,age,education,major,years_exp,compensation,'
                    'python,r,sql'.split(',')]
        )


def topn(ser, n=5, default='other'):
    """
    Replace all values in a Pandas Series that are not among
    the top `n` most frequent values with a default value.

    This function takes a Pandas Series and returns a new
    Series with the values replaced as described above. The
    top `n` most frequent values are determined using the
    `value_counts` method of the input Series.

    Parameters
    ----------
    ser : pd.Series
        The input Series.
    n : int, optional
        The number of most frequent values to keep. The
        default value is 5.
    default : str, optional
        The default value to use for values that are not among
        the top `n` most frequent values. The default value is
        'other'.

    Returns
```

```
    -------
    pd.Series
        The modified Series with the values replaced.
    """
    counts = ser.value_counts()
    return ser.where(ser.isin(counts.index[:n]), default)
```

I created the `TweakKagTransformer` class to wrap `tweak_kag` so I could embed the cleanup logic into the pipeline functionality of Scikit-Learn.

It is not a requirement to use pipelines to use XGBoost. To create a model with XGBoost suitable for classification, you need a matrix with rows of training data with columns representing the features of the data. Generally, this is called `X` in most documentation and examples. You will also need a column with labels for each row, commonly named `y`.

The raw data might look like this:

```
         Q2     Q3                 Q4  \
587   25-29  India    Master's degree
3065  22-24  India  Bachelor's degree
8435  22-24  India    Master's degree

                                                   Q5
587   Information technology, networking, or system ...
3065      Computer science (software engineering, etc.)
8435                                              Other
```

We need to get `X` into looking like this:

```
      age  education  Q3_India  major_cs
587    25       18.0         1         0
3065   22       16.0         1         1
8435   22       18.0         1         0
```

Note

The capitalization on `X` is meant to indicate that the data is a matrix and two-dimensional, while the lowercase `y` is a one-dimensional vector. If you are familiar with linear algebra, you might recognize these conventions, capital letters for matrices and lowercase for vectors.

The transformer class subclasses the `BaseEstimator` and `TransformerMixin` classes. These classes require that we implement the `.fit` and `.transform` methods, respectively. The `.fit` method returns the class instance. The `.transform` method leverages the logic in the `tweak_kag` function.

```
from feature_engine import encoding, imputation
from sklearn import base, pipeline


class TweakKagTransformer(base.BaseEstimator,
    base.TransformerMixin):
    """
    A transformer for tweaking Kaggle survey data.
```

```
    This transformer takes a Pandas DataFrame containing
    Kaggle survey data as input and returns a new version of
    the DataFrame. The modifications include extracting and
    transforming certain columns, renaming columns, and
    selecting a subset of columns.

    Parameters
    ----------
    ycol : str, optional
        The name of the column to be used as the target variable.
        If not specified, the target variable will not be set.

    Attributes
    ----------
    ycol : str
        The name of the column to be used as the target variable.
    """

    def __init__(self, ycol=None):
        self.ycol = ycol

    def transform(self, X):
        return tweak_kag(X)

    def fit(self, X, y=None):
        return self
```

The `get_rawX_y` function will take the original data and return an `X` DataFrame and a `y` Series ready to feed into our pipeline for further cleanup. It uses the `.query` method of Pandas to limit the rows to only those located in the US, China, or India, and respondents that had the job title of *Data Scientist* or *Software Engineer*.

Below that, a pipeline is stored in the variable `kag_pl`. The pipeline will process the data by calling the `TweakKagTransformer`. Then it will perform *one hot encoding* on the *Q1*, *Q3*, and *major* columns using the Feature Engine library. Finally, it will use the `imputation` library to fill in missing numeric values in the *education* and *year_exp* columns. It does that by filling in the missing values with the median values.

```
def get_rawX_y(df, y_col):
    raw = (df
           .query('Q3.isin(["United States of America", "China", "India"]) '
              'and Q6.isin(["Data Scientist", "Software Engineer"])')
          )
    return raw.drop(columns=[y_col]), raw[y_col]


## Create a pipeline
kag_pl = pipeline.Pipeline(
    [('tweak', TweakKagTransformer()),
     ('cat', encoding.OneHotEncoder(top_categories=5, drop_last=True,
           variables=['Q1', 'Q3', 'major'])),
     ('num_impute', imputation.MeanMedianImputer(imputation_method='median',
```

```
            variables=['education', 'years_exp']))]
)
```

Let's run the code.

We will to create a *training set* and a *validation set* (also called test set or holdout set) using the scikit-learn `model_selection.train_test_split` function. This function will withhold 30% of the data into the test dataset (see `test_size=.3`). This will let us train the model on 70% of data, then use the other 30% to simulate unseen data and let us experiment with how the model might perform on data it hasn't seen before.

Note

The `stratify` parameter of the `train_test_split` function is important because it ensures that the proportion of different classes found in the labels of in the dataset is maintained in both the training and test sets. This is particularly useful when working with imbalanced datasets, where one class may be under-represented. Without stratification, the training and test sets may not accurately reflect the distribution of classes in the original dataset, which can lead to biased or unreliable model performance.

For example, if we have a binary classification problem, where the dataset has 80% class A and 20% class B, if we stratify, we will split the data into training and test sets. We will have 80% class A and 20% class B in both sets, which helps to ensure that the classifier will generalize well to unseen data. If we don't stratify, it is possible to get all of class B in the test set, hampering the ability of the model to learn class B.

To use stratification, you pass in the labels to the `stratify` parameter.

Once we have split the data, we can feed it into our pipeline. There are two main methods that we want to be aware of in our pipeline, `.fit` and `.transform`.

The `.fit` method is used to train a model on the training data. It takes in the training data as input and uses that data to *learn* the parameters of the model. For example, in the case of the pipeline that includes our tweaking function, one-hot encoding, and imputation step, the `.fit` method runs each of those steps to learn how to perform the transformation in a consistent manner on new data. We generally use the `.fit` method for the side effect of learning, it does not return transformed data.

The `.transform` method is used to transform the input data according to the steps of the pipeline. The method takes the input data and applies the transformations specified in the pipeline to it (it does not learn any parameters). This method is typically used on test data to apply the same preprocessing steps that were learned on the training data. In our case, we also want to run it on the training data so that it is prepped for use in models. It should return data ready for use with a machine learning model.

The `.fit_transform` method is a combination of the `.fit` and `.transform` methods. It first fits the model on the training data and then applies the learned transformations to the input data, returning the transformed data. This method is useful when we want to learn the parameters of the pipeline on the training data and then apply these learned parameters to the training data all at once. We only want to run this on the training data. Then after we have fit the pipeline on the training data, we will call `.transform` on the testing data.

The output of calling `.fit_transform` and `.transform` on the pipeline is a Pandas DataFrame suitable for training an XGBoost model.

```
>>> from sklearn import model_selection
>>> kag_X, kag_y = get_rawX_y(raw, 'Q6')

>>> kag_X_train, kag_X_test, kag_y_train, kag_y_test = \
```

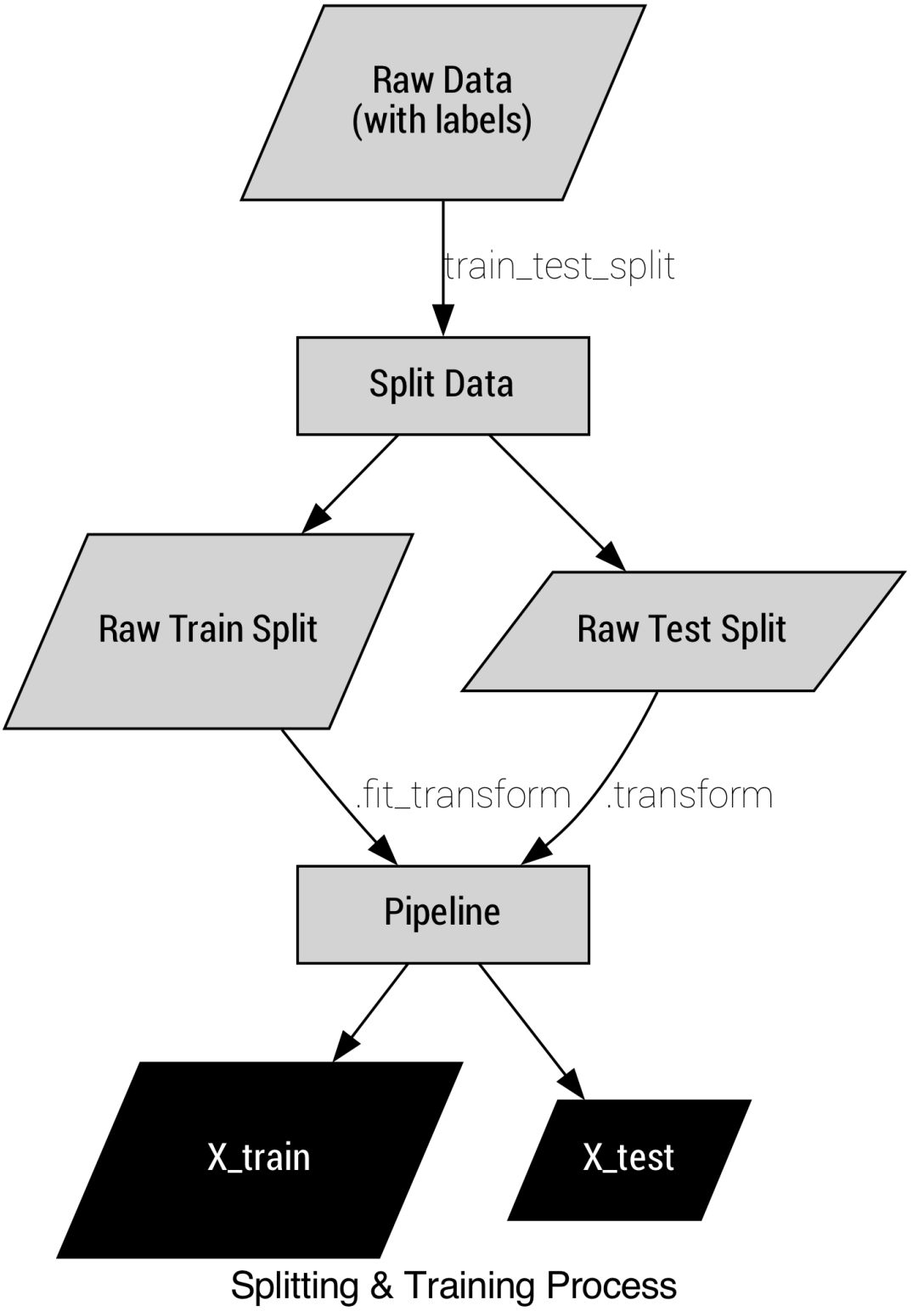


Figure 2.1: Process for splitting data and transforming data with a pipeline.

```
...     model_selection.train_test_split(
...         kag_X, kag_y, test_size=.3, random_state=42, stratify=kag_y)

>>> X_train = kag_pl.fit_transform(kag_X_train, kag_y_train)
>>> X_test = kag_pl.transform(kag_X_test)
>>> print(X_train)
       age  education  years_exp  ...  major_other  major_eng  major_stat
587     25       18.0        4.0  ...            1          0           0
3065    22       16.0        1.0  ...            0          0           0
8435    22       18.0        1.0  ...            1          0           0
3110    40       20.0        3.0  ...            1          0           0
16372   45       12.0        5.0  ...            1          0           0
...    ...        ...        ...  ...          ...        ...         ...
16608   25       16.0        2.0  ...            0          0           0
7325    18       16.0        1.0  ...            0          0           0
21810   18       16.0        2.0  ...            0          0           0
4917    25       18.0        1.0  ...            0          0           1
639     25       18.0        1.0  ...            0          0           0

[2110 rows x 18 columns]
```

At this point, we should be ready to create a model so we can make predictions. Remember, our goal is to predict whether an individual is a *Software Engineer* or *Data Scientist*. Here are the labels for the training data:

```
>>> kag_y_train
587      Software Engineer
3065        Data Scientist
8435        Data Scientist
3110        Data Scientist
16372    Software Engineer
              ...
16608    Software Engineer
7325     Software Engineer
21810       Data Scientist
4917        Data Scientist
639         Data Scientist
Name: Q6, Length: 2110, dtype: object
```

Our data is in pretty good shape for machine learning. Before we jump into that we want to explore the data a little more. We will do that in the next chapter. Data exploration is a worthwhile effort to create better models.

2.3 Summary

In this chapter, we loaded the data. Then we adapted a function that cleans up the data with Pandas into a Scikit-learn pipeline. We used that to prepare the data for machine learning and then split the data into training and testing data.

2.4 Exercises

1. Import the pandas library and use the `read_csv` function to load in a CSV file of your choice.
2. Use the `.describe` method to generate summary statistics for all numeric columns in the DataFrame.
3. Explain how you can convert categorical string data into numbers.

Chapter 3

Exploratory Data Analysis

In this chapter, we will explore the Kaggle data before creating models from it. This is called *Exploratory Data Analysis* or EDA. My goal with EDA is to understand the data I'm dealing with by using summary statistics and visualizations. This process provides insights into what data we have and what the data can tell us.

The exploration step is the most crucial step in data modeling. If you understand the data, you can better model it. You will also understand the relationships between the columns and the target you are trying to predict. This will also enable *feature engineering*, creating new features to drive the model to perform better.

I will show some of my favorite techniques for exploring the data. Generally, I use Pandas to facilitate this exploration. For more details on leveraging Pandas, check out the book *Effective Pandas*.

3.1 Correlations

The correlation coefficient is a useful summary statistic for understanding the relationship between features. There are various ways to calculate this measure. Generally, the *Pearson Correlation Coefficient* metric is used. The calculation for this metric assumes a linear relationship between variables. I often use the *Spearman Correlation Coefficient* which correlates the ranks or monotonicity (and doesn't make the assumption of linearity). The values for both coefficients range from -1 to 1. Higher values indicate that as one value increases, the other value increases. A correlation of 0 means that as one variable increases, the other doesn't respond. A negative correlation indicates that one variable goes down as the other goes up.

I like to follow up correlation exploration with a scatter plot to visualize the relationship.

Here is the Pandas code to create a correlation dataframe. I'll step through what each line does:

- Start with the *X_train* data
- Create a *data_scientist* column (so we can see correlations to it)
- Create a correlation dataframe using the Spearman metric
- Start styling the result (to indicate the best correlations visually)
- Add a background gradient using a diverging colormap (red to white to blue, *RdBu*), pinning the minimum value of -1 to red and maximum value of 1 to blue
- Make the index sticky, so it stays on the screen during horizontal scrolling

```
(X_train
 .assign(data_scientist = kag_y_train == 'Data Scientist')
 .corr(method='spearman')
```

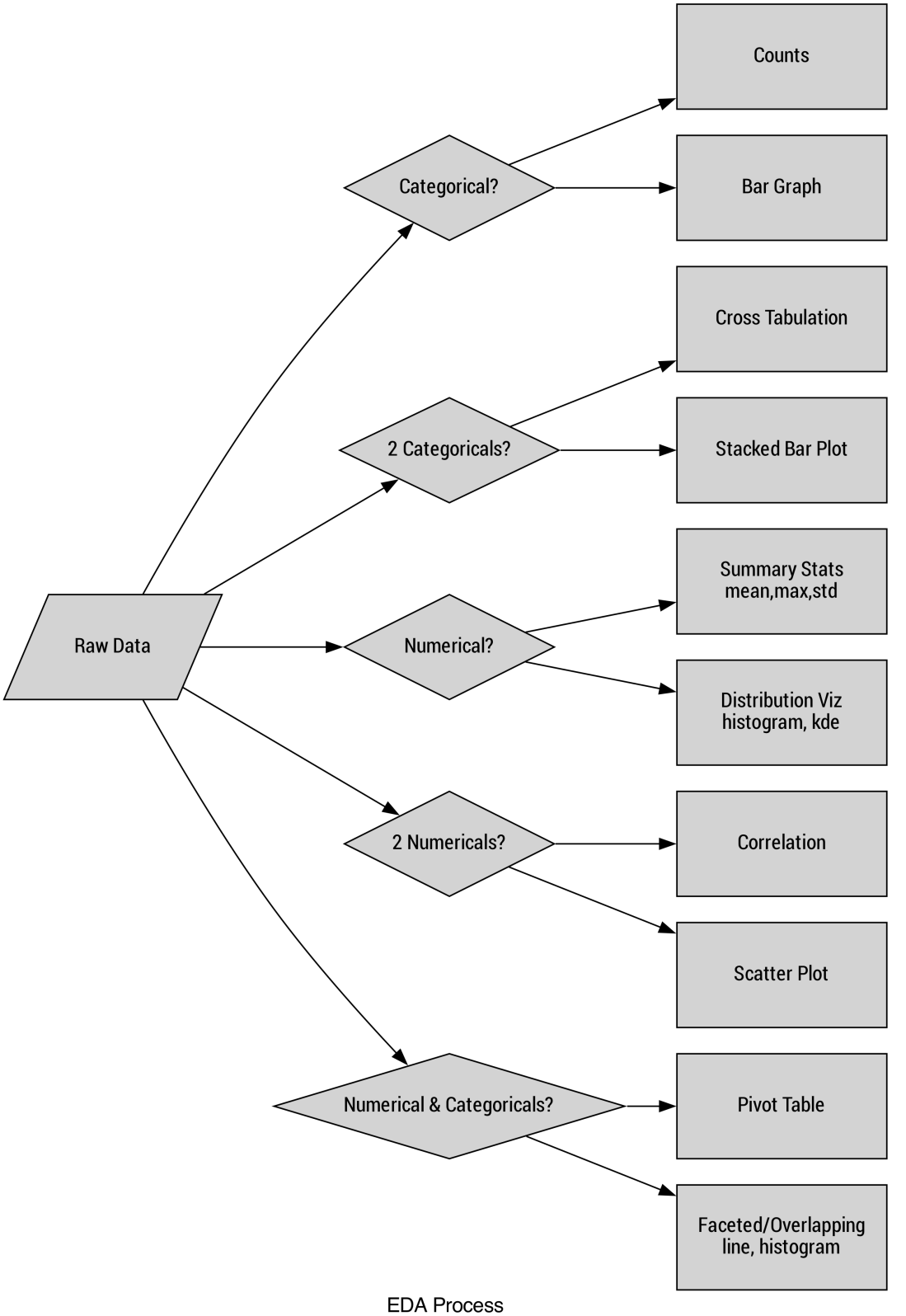


Figure 3.1: Flowchart for my process for basic EDA. I include numerical summaries and visual options.

```
 .style
 .background_gradient(cmap='RdBu', vmax=1, vmin=-1)
 .set_sticky(axis='index')
)
```

```
(X_train
 .assign(data_scientist = kag_y_train == 'Data Scientist')
 .corr(method='spearman')
 .style
 .background_gradient(cmap='RdBu', vmax=1, vmin=-1)
 .set_sticky(axis='index')
)
```

	age	education	years_exp	compensation	python	r	sql	Q1_Male
age	1.000000	0.345717	0.644863	0.451349	0.046651	0.127667	0.134325	0.014277
education	0.345717	1.000000	0.098849	0.241393	0.060259	0.151244	0.066620	-0.079534
years_exp	0.644863	0.098849	1.000000	0.255164	-0.055408	0.012090	0.050181	0.083624
compensation	0.451349	0.241393	0.255164	1.000000	0.165070	0.171485	0.201336	0.000967
python	0.046651	0.060259	-0.055408	0.165070	1.000000	0.183665	0.287957	0.039944
r	0.127667	0.151244	0.012090	0.171485	0.183665	1.000000	0.276862	-0.066982
sql	0.134325	0.066620	0.050181	0.201336	0.287957	0.276862	1.000000	-0.026692
Q1_Male	0.014277	-0.079534	0.083624	0.000967	0.039944	-0.066982	-0.026692	1.000000
Q1_Female	-0.033693	0.064777	-0.097564	0.002912	-0.039207	0.068157	0.032639	-0.957007
Q1_Prefer not to say	0.051232	0.060984	0.034470	-0.035554	-0.006729	0.004831	-0.010814	-0.222963
Q1_Prefer to self-describe	0.041474	0.002701	0.016784	0.049881	-0.000553	-0.004454	-0.018499	-0.094664
Q3_United States of America	0.371123	0.272790	0.117912	0.616759	0.052339	0.178682	0.166587	-0.144665
Q3_India	-0.363626	-0.273408	-0.136666	-0.556154	-0.043101	-0.105625	-0.107382	0.116961
Q3_China	-0.023384	-0.006441	0.029630	-0.123281	-0.017642	-0.133029	-0.108406	0.052566
major_cs	-0.223529	-0.216724	-0.046551	-0.225934	-0.095136	-0.205538	-0.130081	0.062605
major_other	0.201910	0.164771	0.018267	0.241580	0.093887	0.139994	0.109862	-0.071139
major_eng	0.006262	-0.001043	0.004264	-0.060256	0.008640	-0.022082	-0.012964	0.028492
major_stat	0.073864	0.121294	0.042917	0.109100	0.013474	0.162942	0.073906	-0.039544
data_scientist	0.075440	0.266717	-0.145461	0.174434	0.121741	0.329124	0.117941	-0.091126

Figure 3.2: Spearman correlation coefficient of features in dataset. We want to look for the darkest blue and red values ignoring the diagonal.

3.2 Bar Plot

If we inspect the columns that correlate with *data_scientist*, the most significant feature is *r* (with a value of 0.32). Let's explore what is going on there. The *r* column is an *indicator column* that is 1 or 0, depending on if the sample uses the R language. We could use a scatter plot to view the relationship, but it will be less compelling because there are only two values for data scientist, the values will all land on top of one of the two options. Instead, we will use an unstacked bar plot.

Here's what the code is doing line by line:

- Start with the *X_train* dataframe

- Create a *data_scientist* column
- Group by the *r* column
- Look only at the *data_scientist* column
- Get the counts for each different entry in the *data_scientist* column (which will be represented as a multi-index)
- Pull out the data_scientist level of the multi-index and put it in the columns
- Create a bar plot from each column (*data_scientist* options) for each entry in *r*

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
(X_train
 .assign(data_scientist = kag_y_train)
 .groupby('r')
 .data_scientist
 .value_counts()
 .unstack()
 .plot.bar(ax=ax)
)
```

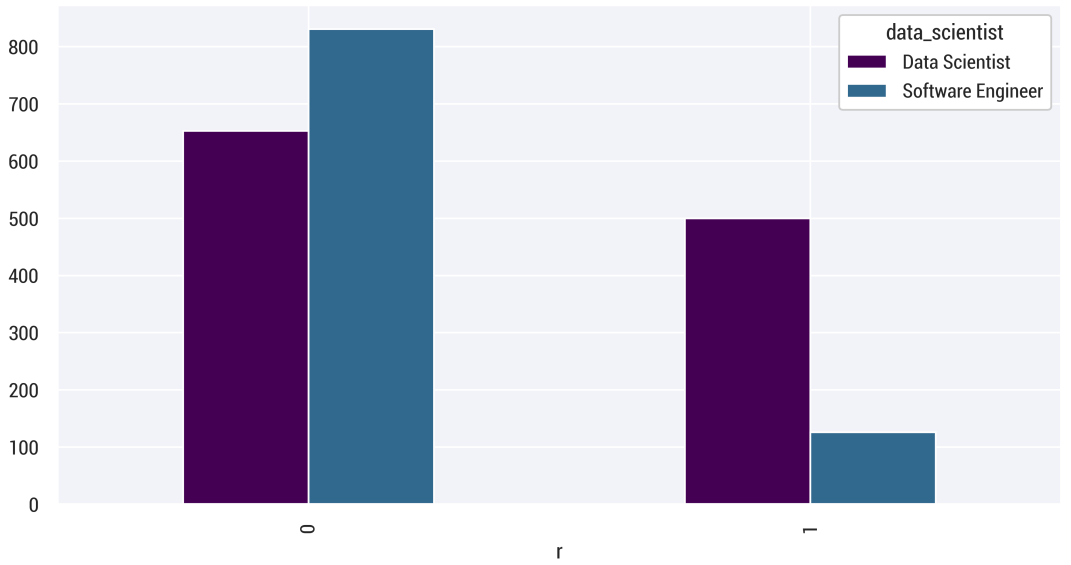


Figure 3.3: Bar plot of R usage for Data Scientists and Software Engineers. The correlation value was positive, 0.32.

It turns out that the column with the most negative correlation to job title (-0.31) was also an indicator column, *major_cs*, indicating whether the respondent studied computer science. Let's explore using the cross-tabulation function (`pd.crosstab`) which provides a helper function for the previous chain of Pandas code.

```
fig, ax = plt.subplots(figsize=(8, 4))
(pd.crosstab(index=X_train['major_cs'],
```

```
            columns=kag_y)
    .plot.bar(ax=ax)
)
```

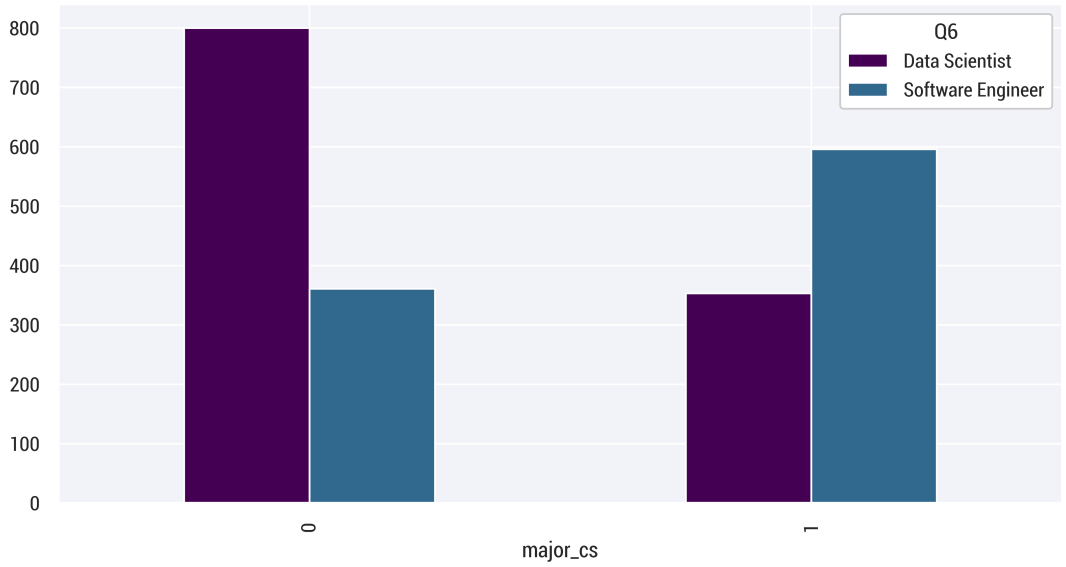


Figure 3.4: Bar plot of majors for Data Scientists and Software Engineers. The correlation value was negative, -0.31.

There is a slight correlation between years of experience and compensation. Let's do a scatter plot of this to see if it helps us understand what is going on.

Here is the code to create a scatter plot:

```
fig, ax = plt.subplots(figsize=(8, 4))
(X_train
 .plot.scatter(x='years_exp', y='compensation', alpha=.3, ax=ax, c='purple')
)
```

This plot is ok, but it is glaringly evident that the data is binned. In the real-world folks would only have exactly five years of experience on one day. Luckily, we set the alpha transparency to see where points pile up on top of each other. But given the plot, it is hard to tell what is going on.

I'm going to use the Seaborn library to create a plot that is more involved. One thing I want to do is add *jitter*. Because our points land on top of each other, it is hard to understand the density. If we shift each one horizontally and vertically (called jitter) by a different random amount, it will aid with that. (This is done with `so.Jitter(x=.5, y=10_000)`

I'm also going to color the points by the title. (`color='title'`)

Finally, we will fit a polynomial line through the points to see the trend. (see `.add(so.Line(), so.PolyFit())`)

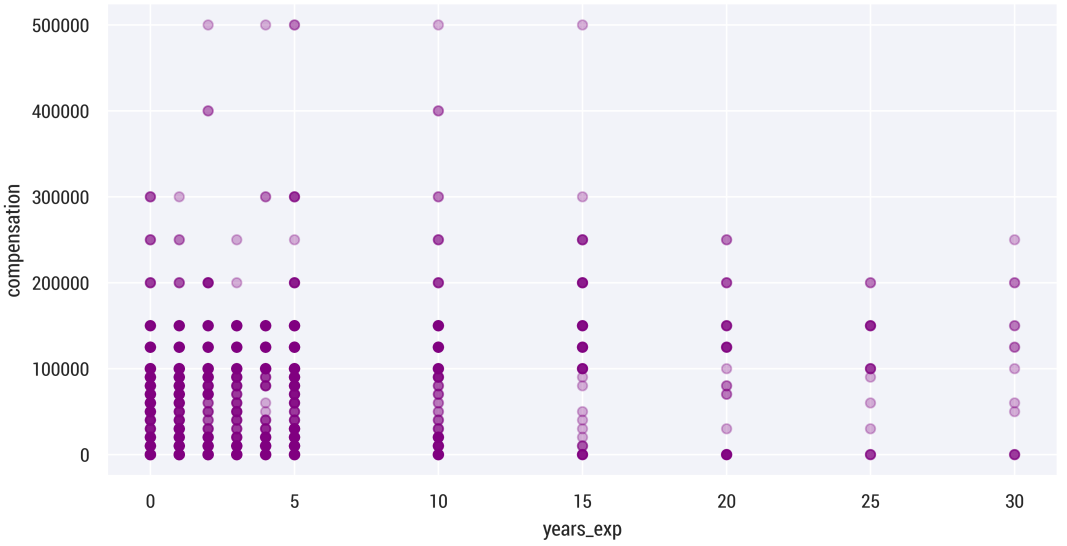


Figure 3.5: Scatter plot showing relationship between compensation and years of experience

```
import seaborn.objects as so
fig = plt.figure(figsize=(8, 4))
(so
 .Plot(X_train.assign(title=kag_y_train), x='years_exp', y='compensation', color='title')
 .add(so.Dots(alpha=.3, pointsize=2), so.Jitter(x=.5, y=10_000))
 .add(so.Line(), so.PolyFit())
 .on(fig)  # not required unless saving to image
 .plot()   # ditto
)
```

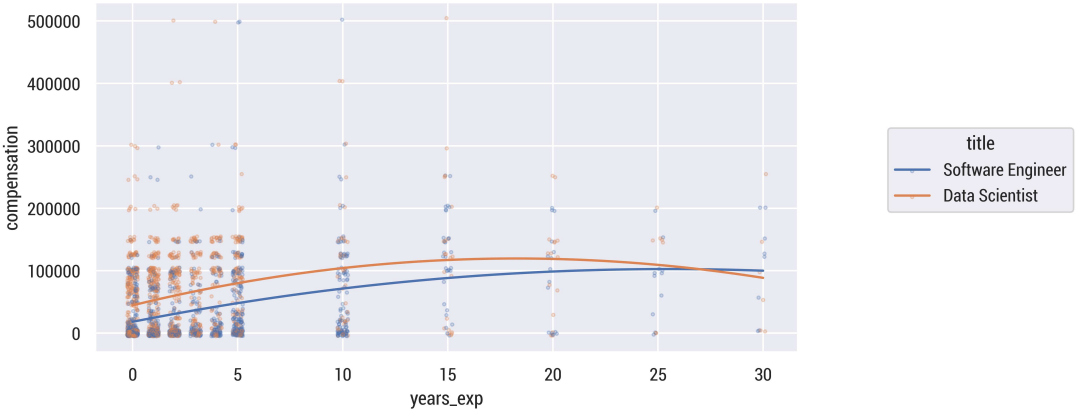


Figure 3.6: Scatter plot showing relationship between compensation and years of experience

This plot is more interesting. It indicates that data scientists tend to earn more than their software engineer counterparts. This hints that compensation might come in useful for determining job titles.

Another point to consider is that this plot lumps together folks from around the world. Different regions are likely to compensate differently. Let's tease that apart as well.

This next plot is cool. The Seaborn objects interface makes it simple to facet the data by country (`.facet('country')`). Also, I want to show all of the data in grey on each plot. If you set `col=None`, it will plot all of the data. However, I want to zoom into to lower left corner because most of the data is found there. I can adjust the tick locations with `.scale(x=so.Continuous().tick(at=[0,1,2,3,4,5]))` and adjust the limit as well with `.limit(y=(-10_000, 200_000), x=(-1, 6))`.

```
fig = plt.figure(figsize=(8, 4))
(so
 .Plot(X_train
       #.query('compensation < 200_000 and years_exp < 16')
       .assign(
         title=kag_y_train,
         country=(X_train
              .loc[:, 'Q3_United States of America': 'Q3_China']
              .idxmax(axis='columns')
             )
       ), x='years_exp', y='compensation', color='title')
 .facet('country')
 .add(so.Dots(alpha=.01, pointsize=2, color='grey' ), so.Jitter(x=.5, y=10_000), col=None)
 .add(so.Dots(alpha=.5, pointsize=1.5), so.Jitter(x=.5, y=10_000))
 .add(so.Line(pointsize=1), so.PolyFit(order=2))
 .scale(x=so.Continuous().tick(at=[0,1,2,3,4,5]))
 .limit(y=(-10_000, 200_000), x=(-1, 6))  # zoom in with this not .query (above)
 .on(fig)  # not required unless saving to image
 .plot()   # ditto
)
```

3.3 Summary

In this chapter, we explored the data to get a feel for what determines whether a respondent is a data scientist. We can use summary statistics to quantify amounts with Pandas. I also like to visualize the data. Typically I will use Pandas and the Seaborn library to make charts to understand relationships between features of the data.

3.4 Exercises

1. Use `.corr` to quantify the correlations in your data.
2. Use a scatter plot to visualize the correlations in your data.
3. Use `.value_counts` to quantify counts of categorical data.
4. Use a bar plot to visualize the counts of categorical data.

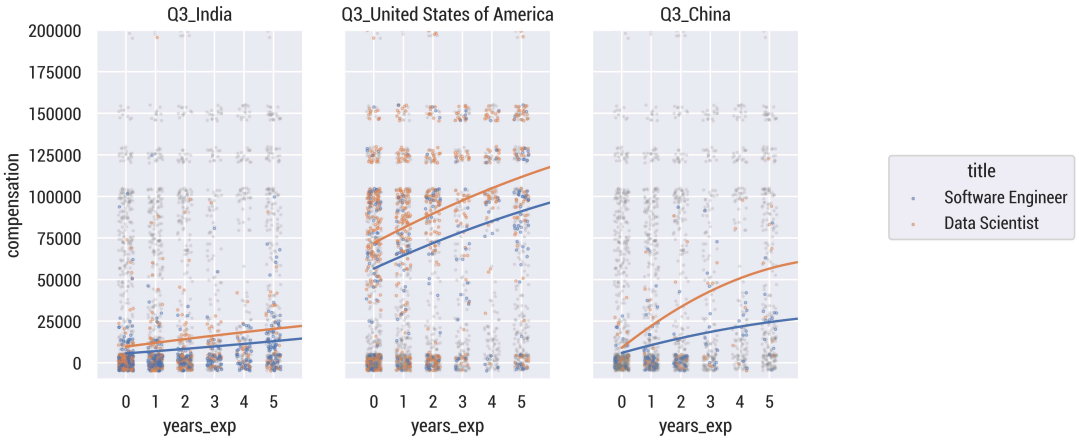


Figure 3.7: Scatter plot showing relationship between compensation and years of experience broken out by country

Chapter 4

Tree Creation

Finally, we are going to make a model! A tree model. Tree-based models are useful for ML because they can handle both categorical and numerical data, and they are able to handle non-linear relationships between features and target variables. Additionally, they provide a clear and interpretable method for feature importance, which can be useful for feature selection and understanding the underlying relationships in the data.

Before we make a model, let's try to understand an algorithm for what makes a good tree model.

At a high level, when we create a tree for classification, the process goes something like this:

- Loop through all of the columns and
  - Find the split point that is best at separating the different labels
  - Create a node with this information
- Repeat the above process for results of each node until each node is pure (or a hyperparameter indicates to stop tree creation)

We need a metric to help determine when our separation is good. One commonly used metric is the *Gini Coefficient*.

4.1 The Gini Coefficient

The Gini coefficient (also known as the Gini index or Gini impurity)[1], quantifies the level of inequality in a frequency distribution. A value of 0 indicates complete equality, with all values being the same, and a value of 1 represents the maximum level of inequality among the values.

The formula for the Gini coefficient is:

$Gini = 1 - \sum_{i=1}^{c} p_i^2$

Where Gini is the Gini index, c is the number of classes, and p_i is the proportion of observations that belong to class i.

In order to understand optimal splitting, we'll simulate a dataset with two classes and one feature. For some values of the features, the classes will overlap—there will be observations from both classes on each side of any split.

We'll find the threshold for which the feature best separates the data into the correct classes.

[1] This coefficient was proposed by an Italian statistician and sociologist Corrado Gini to measure the inequality of wealth in a country. A coefficient of 0 would mean that everyone has the same income. A measure of 1 would indicate that one person has all the income.

Let's explore this by generating two random samples using NumPy. We will label one sample as *Negative* and the other as *Positive*. Here is a visualization of what the distributions look like:

```
import numpy as np
import numpy.random as rn

pos_center = 12
pos_count = 100
neg_center = 7
neg_count = 1000
rs = rn.RandomState(rn.MT19937(rn.SeedSequence(42)))
gini = pd.DataFrame({'value':
    np.append((pos_center) + rs.randn(pos_count),
              (neg_center) + rs.randn(neg_count)),
                     'label':
    ['pos']* pos_count + ['neg'] * neg_count})

fig, ax = plt.subplots(figsize=(8, 4))
_ = (gini
 .groupby('label')
 [['value']]
     .plot.hist(bins=30, alpha=.5, ax=ax, edgecolor='black')
)
ax.legend(['Negative', 'Positive'])
```

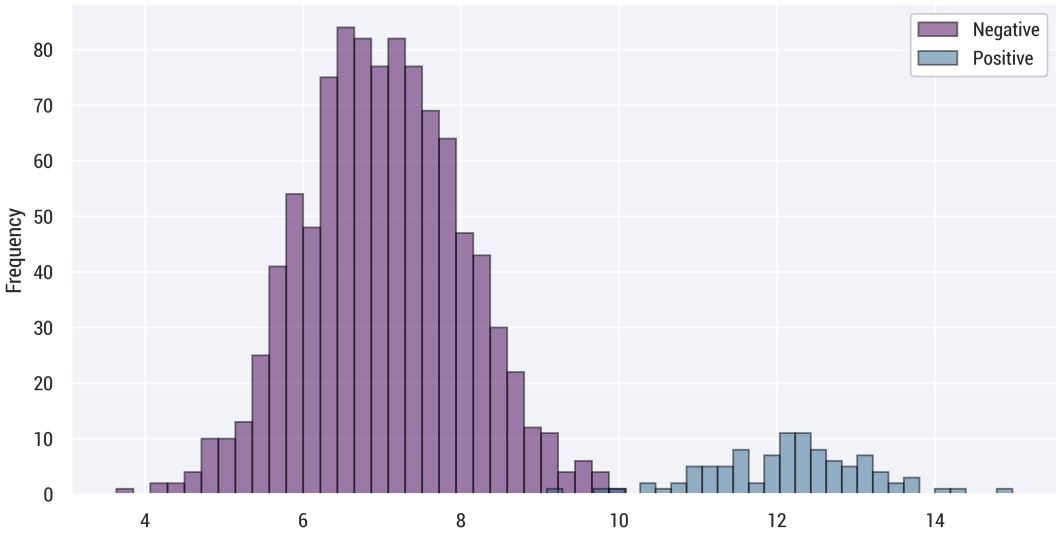


Figure 4.1: Positive and negative distributions for Gini demonstration. Note that there are overlapping values between 9 and 10.

It might be hard to see, but there is some overlap between 9 and 10, where there are both negative and positive examples. I outlined these in black to make them more evident.

Now, let's imagine that we needed to choose a value where we determine whether values are negative or positive. The Gini coefficient is a metric that can help with this task.

To calculate the Gini coefficient for our positive and negative data, we need to consider a point (my function calls it the `split_point`) where we decide whether a value is positive or negative. If we have the actual labels for the data, we can calculate the *true positive*, *false positive*, *true negative*, and *false negative* values. Then we calculate the weighted average of one minus the fraction of true and false positives squared. We also calculate the weighted average of one minus the fraction of true and false negatives squared. The weighted average of these two values is the Gini coefficient.

Here is a function that performs the calculation:

```
def calc_gini(df, val_col, label_col, pos_val, split_point,
              debug=False):
    """
    This function calculates the Gini impurity of a dataset. Gini impurity
    is a measure of the probability of a random sample being classified
    incorrectly when a feature is used to split the data. The lower the
    impurity, the better the split.

    Parameters:
    df (pd.DataFrame): The dataframe containing the data
    val_col (str): The column name of the feature used to split the data
    label_col (str): The column name of the target variable
    pos_val (str or int): The value of the target variable that represents
        the positive class
    split_point (float): The threshold used to split the data.
    debug (bool): optional, when set to True, prints the calculated Gini
        impurities and the final weighted average

    Returns:
    float: The weighted average of Gini impurity for the positive and
        negative subsets.
    """
    ge_split = df[val_col] >= split_point
    eq_pos = df[label_col] == pos_val
    tp = df[ge_split & eq_pos].shape[0]
    fp = df[ge_split & ~eq_pos].shape[0]
    tn = df[~ge_split & ~eq_pos].shape[0]
    fn = df[~ge_split & eq_pos].shape[0]
    pos_size = tp+fp
    neg_size = tn+fn
    total_size = len(df)
    if pos_size == 0:
        gini_pos = 0
    else:
        gini_pos = 1 - (tp/pos_size)**2 - (fp/pos_size)**2
    if neg_size == 0:
        gini_neg = 0
    else:
        gini_neg = 1 - (tn/neg_size)**2 - (fn/neg_size)**2
    weighted_avg = gini_pos * (pos_size/total_size) + \
                   gini_neg * (neg_size/total_size)
```

```
    if debug:
        print(f'{gini_pos=:.3} {gini_neg=:.3} {weighted_avg=:.3}')
    return weighted_avg
```

Let's choose a split point and calculate the Gini coefficient (the *weighted_avg* value).

```
>>> calc_gini(gini, val_col='value', label_col='label', pos_val='pos',
...           split_point=9.24, debug=True)
gini_pos=0.217 gini_neg=0.00202 weighted_avg=0.0241
0.024117224644432264
```

With the function in hand, let's loop over the possible values for the split point and plot out the coefficients for a given split. If we go too low, we get too many false positives. Conversely, if we set the split point too high, we get too many false negatives, and somewhere in between lies the lowest Gini coefficient.

```
values = np.arange(5, 15, .1)
ginis = []
for v in values:
    ginis.append(calc_gini(gini, val_col='value', label_col='label',
                           pos_val='pos', split_point=v))
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(values, ginis)
ax.set_title('Gini Coefficient')
ax.set_ylabel('Gini Coefficient')
ax.set_xlabel('Split Point')
```

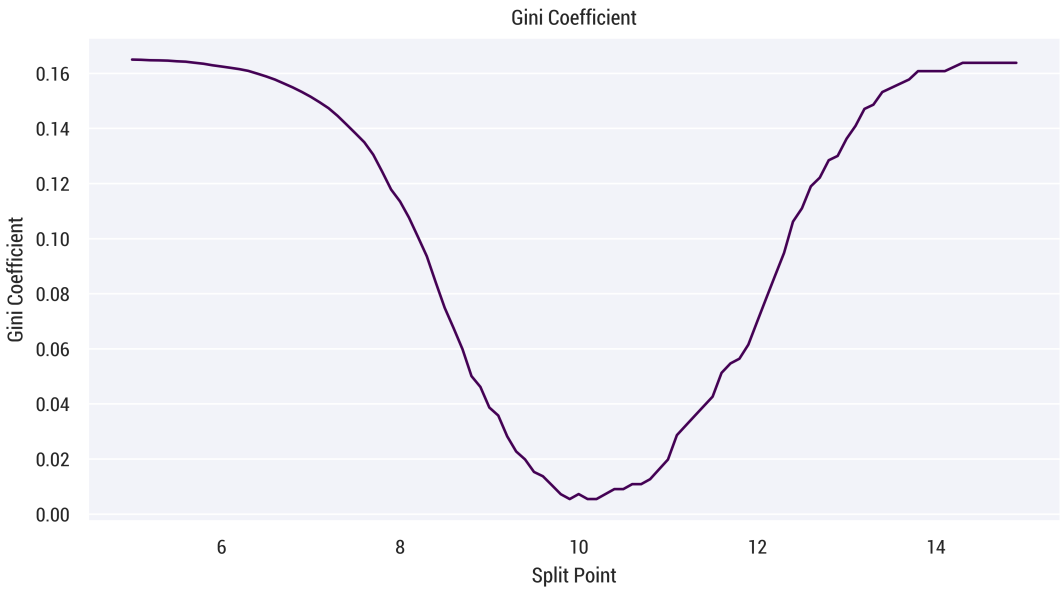


Figure 4.2: Gini values over different split points. Notice that the lowest value is around 10.

Armed with this graph, somewhere around 10 provides the lowest Gini coefficient. Let's look at the values around 10.

```
>>> pd.Series(ginis, index=values).loc[9.5:10.5]
9.6     0.013703
9.7     0.010470
9.8     0.007193
9.9     0.005429
10.0    0.007238
10.1    0.005438
10.2    0.005438
10.3    0.007244
10.4    0.009046
10.5    0.009046
dtype: float64
```

This code verifies that the split point at 9.9 will minimize the Gini coefficient:

```
>>> print(pd.DataFrame({'gini':ginis, 'split':values})
...  .query('gini <= gini.min()')
... )
        gini  split
49  0.005429    9.9
```

4.2 Coefficients in Trees

Now let's make a simple tree that only has a single node. We call this a *decision stump* (because it has no branches).

We can use a `DecisionTreeClassifier` from scikit-learn. We will call the `.fit` method to fit the training data to the labels. Remember, we pass in a dataframe for the features and a series for the labels. (We can also pass in NumPy data structures but I find those inconvenient for tabular work.)

```
from sklearn import tree
stump = tree.DecisionTreeClassifier(max_depth=1)
stump.fit(gini[['value']], gini.label)
```

A valuable decision tree feature is that you can visualize what they do. Below, we will create a visualization of the decision tree.

It shows a single decision at the top box (or node). It checks if the value less than or equal to 9.708. If that decision is true, we jump to the left child, the negative label. Otherwise, we use the right child, the positive label.

You can see that the base Gini value (given in the top box) is our starting Gini considering that we labeled everything as positive.

```
fig, ax = plt.subplots(figsize=(8, 4))
tree.plot_tree(stump, feature_names=['value'],
               filled=True,
               class_names=stump.classes_,
               ax=ax)
```

If you calculate the weighted average of the Gini coefficients in the leaf nodes, you see a value very similar to the minmum value we calculated above:

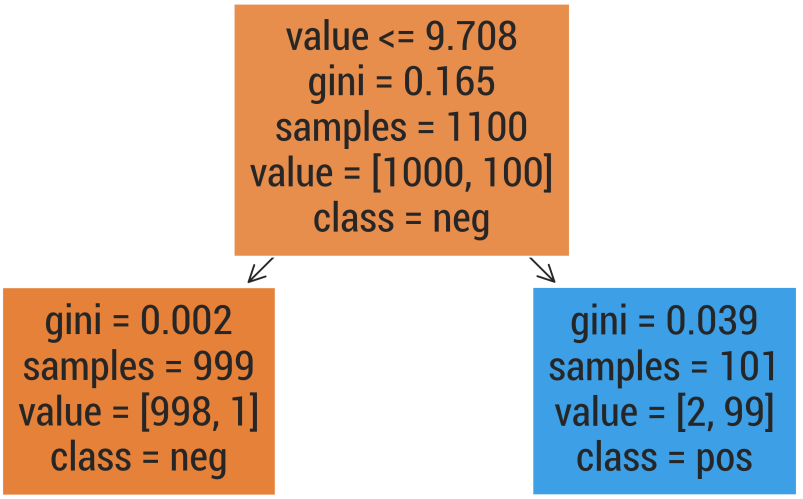


Figure 4.3: Export of decision stump.

```
>>> gini_pos = 0.039
>>> gini_neg = 0.002
>>> pos_size = 101
>>> neg_size = 999
>>> total_size = pos_size + neg_size
>>> weighted_avg = gini_pos * (pos_size/total_size) + \
...                gini_neg * (neg_size/total_size)
>>> print(weighted_avg)
0.005397272727272727
```

XGBoost doesn't use Gini, but Scikit-Learn does by default. XGBoost goes through a tree-building process. After training a tree, there are two statistics g_i and h_i. They are similar to Gini but also represent the behavior of the loss function. The gradient (g_i) represents the first derivative of the loss function with respect to the predicted value for each instance, while the second derivative (h_i) represents the curvature of the loss function.

Let's make a stump in XGBoost by limiting the *max_depth* and *n_estimators* parameters:

```
import xgboost as xgb
xg_stump = xgb.XGBClassifier(n_estimators=1, max_depth=1)
xg_stump.fit(gini[['value']], (gini.label== 'pos'))
```

The `.plot_tree` method will let us visualize the split point for the stump.

```
xgb.plot_tree(xg_stump, num_trees=0)
```

Note

Because I want to be able to set the fonts in this plot, I created my own function, `my_dot_export`.

```
import subprocess
def my_dot_export(xg, num_trees, filename, title='', direction='TB'):
    """Exports a specified number of trees from an XGBoost model as a graph
    visualization in dot and png formats.

    Args:
        xg: An XGBoost model.
        num_trees: The number of tree to export.
        filename: The name of the file to save the exported visualization.
        title: The title to display on the graph visualization (optional).
        direction: The direction to lay out the graph, either 'TB' (top to
            bottom) or 'LR' (left to right) (optional).
    """
    res = xgb.to_graphviz(xg, num_trees=num_trees)
    content = f'''    node [fontname = "Roboto Condensed"];
    edge [fontname = "Roboto Thin"];
    label = "{title}"
    fontname = "Roboto Condensed"
    '''
    out = res.source.replace('graph [ rankdir=TB ]',
                             f'graph [ rankdir={direction} ];\n {content}')
    # dot -Gdpi=300 -Tpng -ocourseflow.png courseflow.dot
    dot_filename = filename
    with open(dot_filename, 'w') as fout:
        fout.write(out)
    png_filename = dot_filename.replace('.dot', '.png')
    subprocess.run(f'dot -Gdpi=300 -Tpng -o{png_filename} {dot_filename}'.split())
```

Let's try out the function.

```
my_dot_export(xg_stump, num_trees=0, filename='img/stump_xg.dot', title='A demo stump')
```

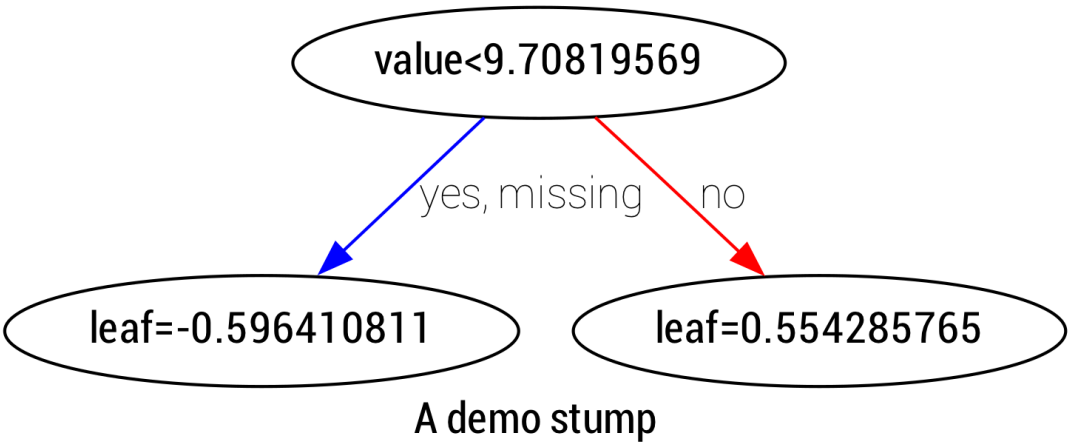


Figure 4.4: Export of xgboost stump.

The split point XGBoost found is very similar to scikit-learn. (The values in the leaves are different, but we will get to those.)

4.3 Another Visualization Tool

The dtreeviz package creates visualizations for decision trees. It provides a clear visualization of tree structure and decision rules, making it easier to understand and interpret the model.

```
import dtreeviz
viz = dtreeviz.model(xg_stump, X_train=gini[['value']],
                     y_train=gini.label=='pos',
    target_name='positive',
    feature_names=['value'], class_names=['negative', 'positive'],
    tree_index=0)
viz.view()
```

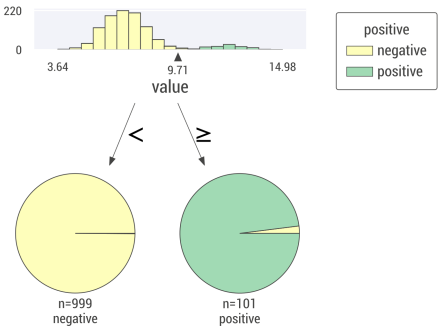


Figure 4.5: Decision stump visualized by dtreeviz package. Note that it indicates the split point in the histogram. It also shows a pie chart for each leaf indicating the fraction of labels.

This visualization shows the distributions of the data as histograms. It also labels the split point. Finally, it shows what the ratios of the predictions are as pie charts.

4.4 Summary

The Gini coefficient is a measure of inequality used in economics and statistics. It is typically used to measure income inequality, but can also be applied to other variables such as wealth. The Gini coefficient takes on a value between 0 and 1, with 0 representing perfect equality (where everyone has the same income) and 1 representing perfect inequality (where one person has all the income and everyone else has none).

4.5 Exercises

1. What is the Gini coefficient?
2. How is the Gini coefficient calculated?
3. How is the Gini coefficient used in Decision trees?
4. What does a stump tell us about the data?

Chapter 5

Stumps on Real Data

By now, you should have an intuition on how a decision tree "decides" values to split on. Decision trees use a greedy algorithm to split on a feature (column) that results in the most "pure" split.

Instead of looking at our simple data with only one column of information, let's use the Kaggle data. We will predict whether someone is a data scientist or software engineer based on how they answered the survey questions. The decision tree should loop over every column and try to find the column and split point that best separates data scientists from software engineers. It will recursively perform this operation, creating a tree structure.

5.1 Scikit-learn stump on real data

Let's use scikit-learn first to make a decision stump on the Kaggle data. Before we do that, consider what this stump is telling us. The first split should feature one of the most critical features because it is the value that best separates the data into classes. If you only had one piece of information, you would probably want the column that the stump splits on.

Let's create the stump and visualize it. We will use our pipeline to prepare the training data. Remember that the pipeline uses Pandas code to clean up the raw data, then we use categorical encoding on the non-numeric columns, and finally, we fill in the missing values for education level and experience.

We will feed the raw data, `kag_X_train`, into the `.fit_transform` method of the pipeline and get out the cleaned-up training data, `X_train`. Then we will create a stum and train it with the `.fit` method:

```
stump_dt = tree.DecisionTreeClassifier(max_depth=1)
X_train = kag_pl.fit_transform(kag_X_train)
stump_dt.fit(X_train, kag_y_train)
```

Let's explore how we decided to split the data. Here is the code to visualize the tree:

```
fig, ax = plt.subplots(figsize=(8, 4))
features = list(c for c in X_train.columns)
tree.plot_tree(stump_dt, feature_names=features,
               filled=True,
               class_names=stump_dt.classes_,
               ax=ax)
```

The column that best separates the two professions is the use of the R programming language. This seems like a sensible choice considering that very few software engineers use the R language.

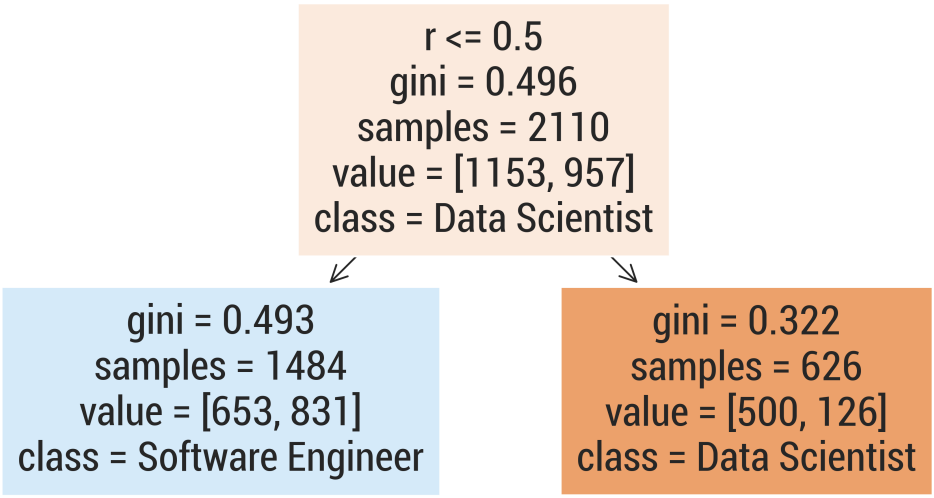


Figure 5.1: Decision stump trained on our Kaggle data. It bases its decision purely on the value in the r column.

We can evaluate the accuracy of our model with the `.score` method. We call this method on the *holdout* set. The holdout set (or validation set), `X_test`, is the portion of the data that is set aside and not used during the training phase, but instead is reserved for testing the model's performance once it has been trained. It allows for a more accurate evaluation of the model's performance against new data, as it is tested on unseen data.

It looks like this model classified 62% of the occupations correctly by just looking at a single column, the *R* column.

```
>>> X_test = kag_pl.transform(kag_X_test)
>>> stump_dt.score(X_test, kag_y_test)
0.6243093922651933
```

Is 62% accuracy good? Bad? The answer to questions like this is generally "it depends". Looking at accuracy on its own is usually not sufficient to inform us of whether a model is "good enough". However, we can use it to compare models. We can create a *baseline* model using the `DummyClassifier`. This classifier just predicts the most common label, in our case `'Data Scientist'`. Not a particularly useful model, but it provides a baseline score that our model should be able to beat. If we can't beat that model, we shouldn't be using machine learning.

Here is an example of using the `DummyClassifier`.

```
>>> from sklearn import dummy
>>> dummy_model = dummy.DummyClassifier()
>>> dummy_model.fit(X_train, kag_y_train)
>>> dummy_model.score(X_test, kag_y_test)
0.5458563535911602
```

The baseline performance is 54% accuracy (which is the percent of values that are `'Data Scientist'`). Our stump accuracy is better than the baseline. Note that being better than the baseline does not qualify a model as "good". But it does indicate that model is potentially useful.

5.2 Decision Stump with XGBoost

XGBoost does not use the Gini calculation to decide how to make decisions. Rather XGBoost uses *boosting* and *gradient descent*. Boosting is using a model and combining it with other models to improve the results. In fact, XGBoost stands for eXtreme Gradient Boosting. The "extreme" part is due to the ability to regularize the result and the various optimizations is has to efficiently create the model.

In this case, the subsequent models are trained from the error of the previous model with the goal to reduce the error. The "gradient descent" part comes in because this process of minimizing the error is specified into an objective function such that the gradient descent algorithm can be applied. The outcome is based on the gradient of the error with respect to the prediction. The objective function combines two parts: training loss and a regularization term. Trees are created by splitting on features that move a small step in the direction of the negative gradient, which moves them closer to the global minimum of the loss function.

There are many parameters for controlling XGBoost. One of them is `n_estimators`, the number of trees. Let's create a stump with XGBoost by setting this value to 1 and see if it performs similarly to the scikit-learn stump.

```
import xgboost as xgb
kag_stump = xgb.XGBClassifier(n_estimators=1, max_depth=1)
kag_stump.fit(X_train, kag_y_train)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[402], line 2
      1 kag_stump = xgb.XGBClassifier(n_estimators=1, max_depth=1)
----> 2 kag_stump.fit(X_train, kag_y_train)
      3 kag_stump.score(X_test, kag_y_test)

...

   1462 if (
   1463     self.classes_.shape != expected_classes.shape
   1464     or not (self.classes_ == expected_classes).all()
   1465 ):
-> 1466     raise ValueError(
   1467         f"Invalid classes inferred from unique values of `y`.  "
   1468         f"Expected: {expected_classes}, got {self.classes_}"
   1469     )
   1471 params = self.get_xgb_params()
   1473 if callable(self.objective):

ValueError: Invalid classes inferred from unique values of `y`.
Expected: [0 1], got ['Data Scientist' 'Software Engineer']
```

We got an error! XGBoost is not happy about our labels. Unlike scikit-learn, XGBoost does not work with string labels. The `kag_y_train` series has strings in it, and XGBoost wants integer values.

```
>>> print(kag_y_train)
587      Software Engineer
3065        Data Scientist
8435        Data Scientist
3110        Data Scientist
16372    Software Engineer
              ...
16608    Software Engineer
7325     Software Engineer
21810       Data Scientist
4917        Data Scientist
639         Data Scientist
Name: Q6, Length: 2110, dtype: object
```

Since there are only two options, we can convert this to integer values by testing whether the string is equal to *Software Engineer*. A `True` or (1) represents Software Engineer and `False` (or 0) is a Data Scientist.

```
>>> print(kag_y_train == 'Software Engineer')
587       True
3065     False
8435     False
3110     False
16372     True
         ...
16608     True
7325      True
21810    False
4917     False
639      False
Name: Q6, Length: 2110, dtype: bool
```

However, rather than using pandas, we will use the `LabelEncoder` class from scikit-learn to convert the labels to numbers. The label encoder stores attributes that are useful when preparing data for future predictions.

```
>>> from sklearn import preprocessing
>>> label_encoder = preprocessing.LabelEncoder()
>>> y_train = label_encoder.fit_transform(kag_y_train)
>>> y_test = label_encoder.transform(kag_y_test)
>>> y_test[:5]
array([1, 0, 0, 1, 1])
```

The label encoder will return 1s and 0s. How do you know which numerical value is which string value? You can ask the label encoder for the `.classes_`. The index of these values is the number that the encoder uses.

```
>>> label_encoder.classes_
array(['Data Scientist', 'Software Engineer'], dtype=object)
```

Because 'Data Scientist' is in index 0 in .classes_, it is the 0 value in the training labels. The 1 represents 'Software Engineer'. This is also called the *positive label*. (It is the positive label because it has the value of 1.)

The label encoder has an .inverse_transform method that reverses the transformation. (This would require more effort if we went with the pandas encoding solution.)

```
>>> label_encoder.inverse_transform([0, 1])
array(['Data Scientist', 'Software Engineer'], dtype=object)
```

Now let's make a stump with XGBoost and check the score.

```
>>> kag_stump = xgb.XGBClassifier(n_estimators=1, max_depth=1)
>>> kag_stump.fit(X_train, y_train)
>>> kag_stump.score(X_test, y_test)
0.6243093922651933
```

It looks like the XGBoost stump performs similarly to the scikit-learn decision tree. Let's visualize what it looks like.

```
my_dot_export(kag_stump, num_trees=0, filename='img/stump_xg_kag.dot',
              title='XGBoost Stump')
```

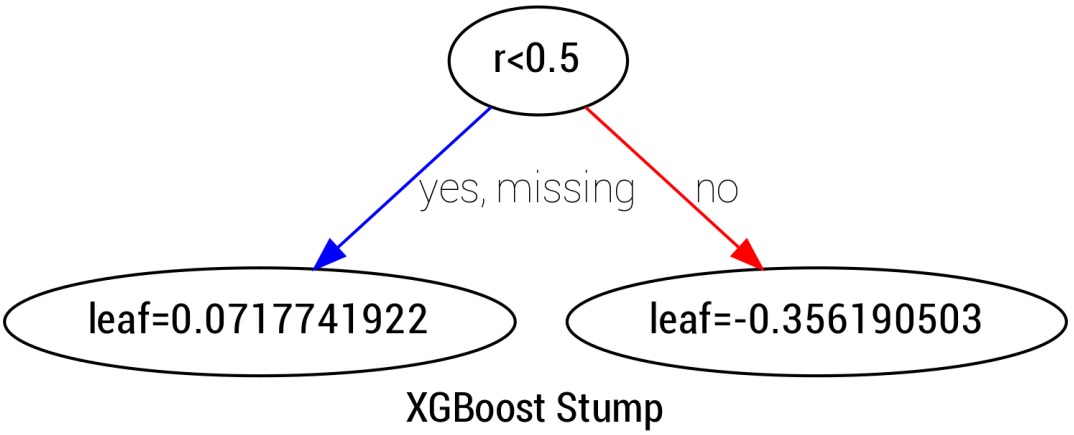


Figure 5.2: Stump export of XGBoost Kaggle model. A positive leaf value means that we predict the positive label.

5.3 Values in the XGBoost Tree

What are the numbers in the leaves of the export? They are probabilities. Well, actually, they are the logarithm of the odds of the positive value. Statisticians call this the *logit*.

When you calculate the inverse logit scores with the values from the nodes in the tree, you get the probability of the prediction. If a survey respondent does not use the R language, the leaf value is .0717741922. The inverse logit of that is .518, or 51.8%. Because this value is greater than .5, we assume the positive label or class of *Software Engineer* (the second value in kag_stump.classes_ is 1, which our label encoder used as the value for Software Engineer).

```
>>> kag_stump.classes_
array([0, 1])
```

```
import numpy as np
def inv_logit(p: float) -> float:
    """
    Compute the inverse logit function of a given value.

    The inverse logit function is defined as:
        f(p) = exp(p) / (1 + exp(p))

    Parameters
    ----------
    p : float
        The input value to the inverse logit function.

    Returns
    -------
    float
        The output of the inverse logit function.
    """
    return np.exp(p) / (1 + np.exp(p))
```

Let's see what probability comes out when we pass in .07177.

```
>>> inv_logit(.0717741922)
0.5179358489487103
```

It looks like it spits out 51.8%.

Conversely, the inverse logit of the right node is .411. Because this number is below .5, we do not classify it as the second option but rather choose *Data Scientist*.

Note

Note that if a user uses R, they have a higher likelihood of being a Data Scientist (1-.41 or 59%), than a Software Engineer who doesn't use R (.518 or 52%). In other words, using R pushes one more toward Data Scientist than not using R pushes one toward Software Engineer.

```
>>> inv_logit(-.3592)
0.41115323716754393
```

Here I plot the inverse logit function. You can see a crossover point at 0 on the x-axis. When the x values are above 0, the y value will be > .5 (Software Engineer). When they are below 0, the values will be < .5 (Data Scientist).

```
fig, ax = plt.subplots(figsize=(8, 4))
vals = np.linspace(-7, 7)
ax.plot(vals, inv_logit(vals))
ax.annotate('Crossover point', (0,.5), (-5,.8), arrowprops={'color':'k'})
ax.annotate('Predict Positive', (5,.6), (1,.6), va='center', arrowprops={'color':'k'})
ax.annotate('Predict Negative', (-5,.4), (-3,.4), va='center', arrowprops={'color':'k'})
```

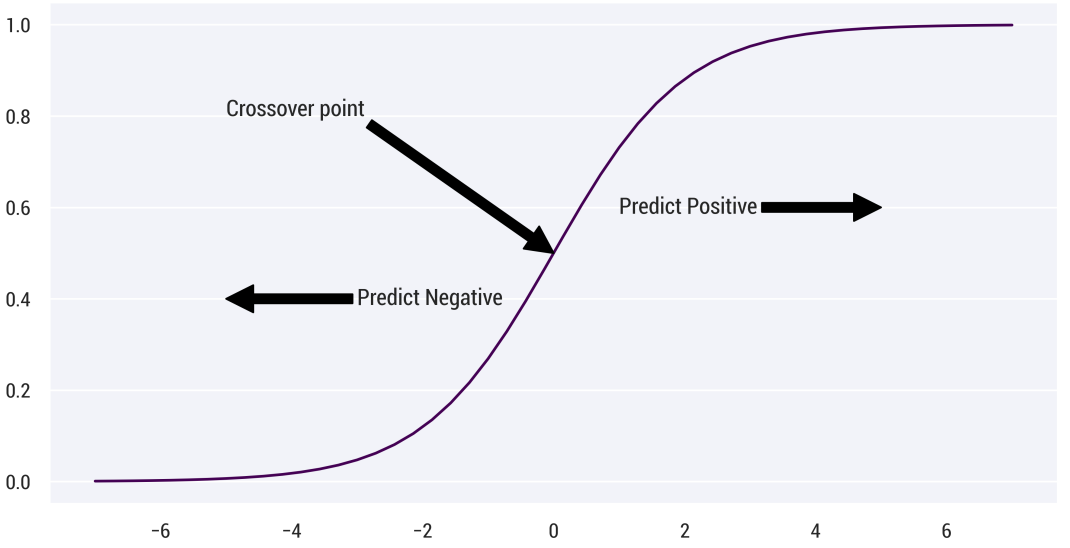


Figure 5.3: The inverse logit function. When the x values are above 0, the y value will be > .5 (Software Engineer). When they are below 0, the values will be < .5 (Data Scientist).

5.4 Summary

In this chapter, we explored making a stump on real data. A decision stump is a simple decision tree with a single split. We showed how we can use the XGBoost algorithm to create a decision stump. We also explored how the values in the leaves of XGBoost models indicate what label to predict.

5.5 Exercises

1. How are decision stumps trained using the XGBoost algorithm?
2. What is the inverse logit function and how is it used in machine learning?
3. How can the output of the inverse logit function be interpreted in terms of label probabilities?

Chapter 6

Model Complexity & Hyperparameters

In this chapter, we will explore the concept of underfit and overfit models. Then we will see how we can use *hyperparameters*, or attributes of the model to change the behavior of the model fitting.

6.1 Underfit

A stump is too simple. Statisticians like to say it has too much *bias*. (I think this term is a little confusing and prefer "underfit".) It has a bias towards simple predictions. In the case of our Kaggle model, the stump only looks at a single column. Perhaps it would perform better if it were able to also consider additional columns after looking at the R column.

When you have an underfit model, you want to make it more complex (because it is too simple). There are a few mechanisms we can use to add complexity:

- Add more features (columns) that have predictive value
- Use a more complex model

These are general strategies that work with most underfit models, not just trees.

Here is our stump. Let's look at the accuracy (as reported by `.score`). It should be representative of an underfit model.

```
>>> underfit = tree.DecisionTreeClassifier(max_depth=1)
>>> X_train = kag_pl.fit_transform(kag_X_train)
>>> underfit.fit(X_train, kag_y_train)
>>> underfit.score(X_test, kag_y_test)
0.6243093922651933
```

Hopefully, we will be able to create a model with better accuracy than 62%.

6.2 Growing a Tree

We showed how to create a stump. How do we fix our underfit stump? You could add more advanced features (or columns) to the training data that better separate the classes. Or we could make the stump more complex by letting the tree grow larger.

Instead of trying arbitrary tactics, generally, we want to proceed in a slightly more organized fashion. One common technique is to measure model accuracy as you let the tree grow deeper (or add additional columns) and see if there is an improvement. You can optimize a scoring function given constraints. Different models can have different scoring functions. You can also create your own. We saw that scikit-learn optimizes Gini and that XGBoosts performs gradient descent on a loss function.

The constraints for the model are called *hyperparameters*. Hyperparameters, at a high level, are knobs that you can use to control the complexity of your model. Remember that our stump was too simple and underfit our data. One way we can add complexity to a tree-based model is to let it grow more nodes. We can use hyperparameters to determine how and when we will add nodes.

6.3 Overfitting

Before optimizing our model, let's look at the other end of the spectrum, an *overfit* model.

An overfit model is too complicated. Again, the statistician would say it has too much variance. I avoid using the term "variance" too much (unless I'm in an interview situation) and like to picture a model that looks at every nook and cranny of the data. When it sees an example that is new, the model compares all of these trivial aspects that the model learned by inspecting the training data, which tends to lead it down the wrong path. Rather than honing in on the right label, the complexity causes it to "vary".

And again, there are general solutions to helping deal with overfitting:

- Simplify or constrain (*regularize*)
- Add more samples (rows of data)

For a tree model, we can prune back the growth so that the leaf nodes are overly specific. This will simplify or constrain the model. Alternatively, we can collect more data, thus forcing the model to have more attempts at finding the important features.

6.4 Overfitting with Decision Trees

Let's make an overfit model with a decision tree. This is quite easy. We just let the model grow until every node is pure (of the same class). We can set `max_depth=None` to overfit the tree, which is the default behavior of `DecisionTreeClassifier`.

After we create the model, let's inspect the accuracy.

```
>>> hi_variance = tree.DecisionTreeClassifier(max_depth=None)
>>> X_train = kag_pl.fit_transform(kag_X_train)
>>> hi_variance.fit(X_train, kag_y_train)
>>> hi_variance.score(X_test, kag_y_test)
0.6629834254143646
```

In this case, the accuracy is 66%. The accuracy of the stump was 62%. It is possible (and likely) that we can get better accuracy than both the stump and the complex model by either simplifying our complex model or adding some complexity to our simple model.

Here is a visualization of the complex model. You can see that the tree is pretty deep. I count 22 levels of nodes. This visualization should help with the intuition of why an overfit tree is bad. It is essentially memorizing the training data. If you feed it data that is not exactly the same as data it has seen before, it will tend to go down the wrong track. (If you think memorizing the data is good, then you don't need a machine learning model because you can just memorize every possible input and create a giant *if statement* for that.)

```
fig, ax = plt.subplots(figsize=(8, 4))
features = list(c for c in X_train.columns)
tree.plot_tree(hi_variance, feature_names=features, filled=True)
```

Let's zoom in on the top of the tree to understand more about what is happening.

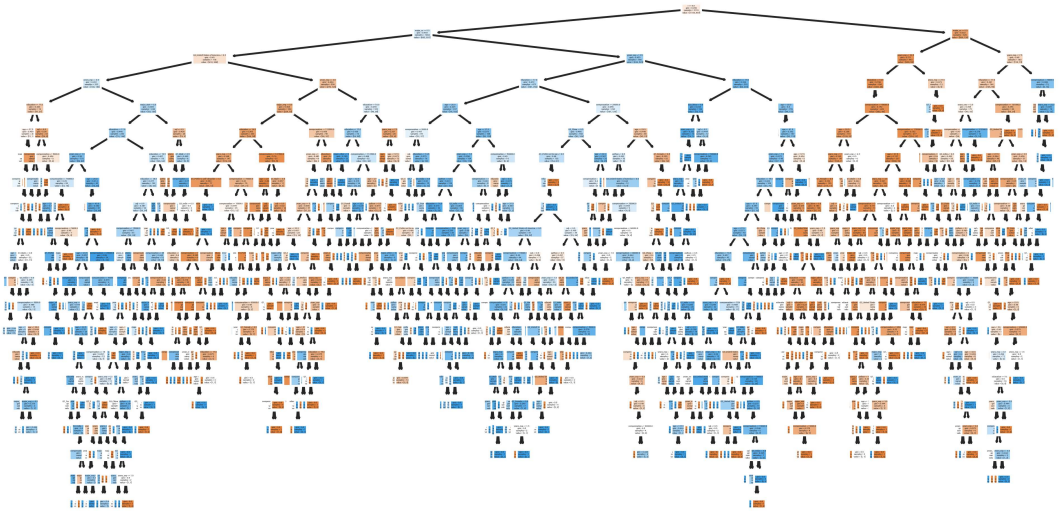


Figure 6.1: An overfit decision tree. This is too complicated and it is essentially memorizing the data that it was trained on. However, because it is very specific to the training data, it will tend to perform worse on data that it hasn't seen before.

```
fig, ax = plt.subplots(figsize=(8, 4))
features = list(c for c in X_train.columns)
tree.plot_tree(hi_variance, feature_names=features, filled=True,
                    class_names=hi_variance.classes_,
                max_depth=2, fontsize=6)
```

The darker orange boxes tend towards more data scientists, and the bluer boxes are software engineers. If we go down the most orange path, at the top node is a person who uses R, who also didn't study computer science, and finally, they have less or equal to 22.5 years of experience. Of folks who fall into this bucket, 390 of the 443 samples were data scientists.

The ability to interpret a decision tree is important and often business-critical. Many companies are willing to sacrifice model performance for the ability to provide a concrete explanation for the behavior of a model. A decision tree makes this very easy. We will see that XGBoost models prove more challenging at delivering a simple interpretation.

6.5 Summary

In this chapter, the concept of underfit and overfit models is explored. Hyperparameters, or attributes of the model, are used to change the behavior of the model fitting. We saw that the `max_depth` hyperparameter can tune the model between simple and complex. An underfit model is too simple, having a bias towards simple predictions. To fix an underfit model, one can add more features or use a more complex model. These general strategies work with most underfit models, not just trees.

On the other hand, an overfit model is too complicated and has too much variance. To deal with overfitting, one can simplify or constrain the model, or add more samples. For a tree model, one can prune back the growth so that the leaf nodes are overly specific or collect

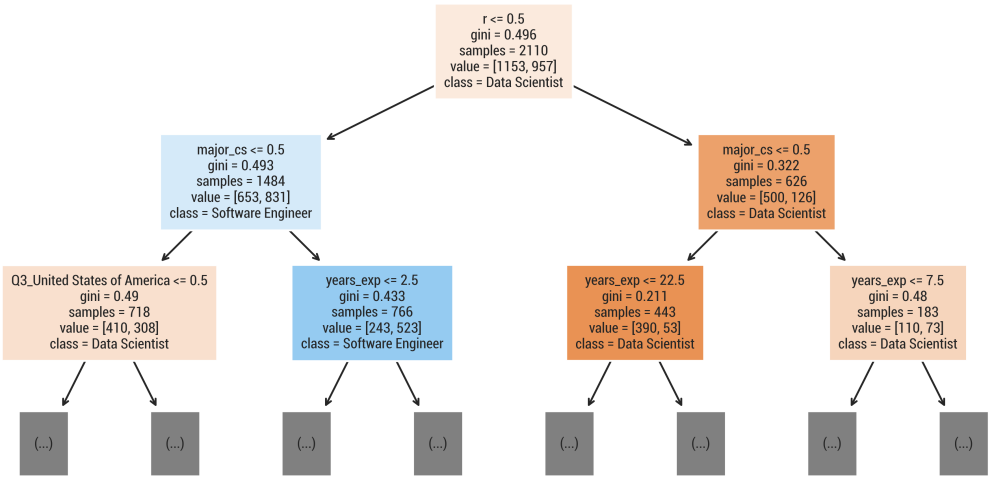


Figure 6.2: Zooming in on an overfit decision tree.

more data. The goal is to find the optimal balance between a model that is too simple and too complex.

Later on, we will see that XGBoost creates decent models out of the box, but it also has mechanisms to tune the complexity. We will also see how you can diagnose overfitting by visualizing model performance.

6.6 Exercises

1. What is underfitting in decision trees and how does it occur?
2. What is overfitting in decision trees and how does it occur?
3. What are some techniques to avoid underfitting in decision trees?
4. What are some techniques to avoid overfitting in decision trees?

Chapter 7

Tree Hyperparameters

Previously, we saw that models might be too complex or too simple. Some of the mechanisms for dealing with model complexity are by creating new columns (to deal with simple models) and adding more examples (to deal with complex models). But there are also levers that we can pull without dealing with data. These levers are *hyperparameters*, levers that change how the model is created, trained, or behaves.

7.1 Decision Tree Hyperparameters

Let's explore the hyperparameters of decision trees. You might wonder why we seem so concerned with decision trees when this book is about XGBoost. That is because decision trees form the basis for XGBoost. If you understand how decision trees work, that will aid your mental model of XGBoost. Also, many of the hyperparameters for decision trees apply to XGBoost at some level.

Many of the hyperparameters impact complexity (or simplicity). Here is a general rule for models based on scikit-learn. Hyperparameters that start with *max_* will make the model more complex when you raise it. On the flip side, those that start with *min_* will make the model simpler if you raise it. (The reverse is also true, they do the reverse when you lower them.)

Here are the hyperparameters for scikit-learn's `DecisionTreeClassifier`:

- `max_depth=None` - Tree depth. The default is to keep splitting until all nodes are pure or there are fewer than `min_samples_split` samples in each node. Range [1-large number].
- `max_features=None` - Amount of features to examine for the split. Default is the number of features. Range [1-number of features].
- `max_leaf_node=None` - Number of leaves in a tree. Range [1-large number].
- `min_impurity_decrease=0` - Split when *impurity* is >= this value. (*Impurity* : 0 - 100% accurate, .3 - 70%). Range [0.0-1.0]
- `min_samples_leaf=1`, - Minimum samples at each leaf. Range [1-large number]
- `min_samples_split=2` - Minimum samples required to split a node. Range [2-large number]
- `min_weight_fraction_leaf=0` - The fraction of the total weights required to be a leaf. Range [0-1]

To use a particular hyperparameter, you provide the parameter in the *constructor* before training a model. The constructor is the method called when a class is created. Then you train a model and measure a performance metric and track what happens to that metric as you change the parameter.

You can explore the parameters of a trained model as well using the `.get_params` method:

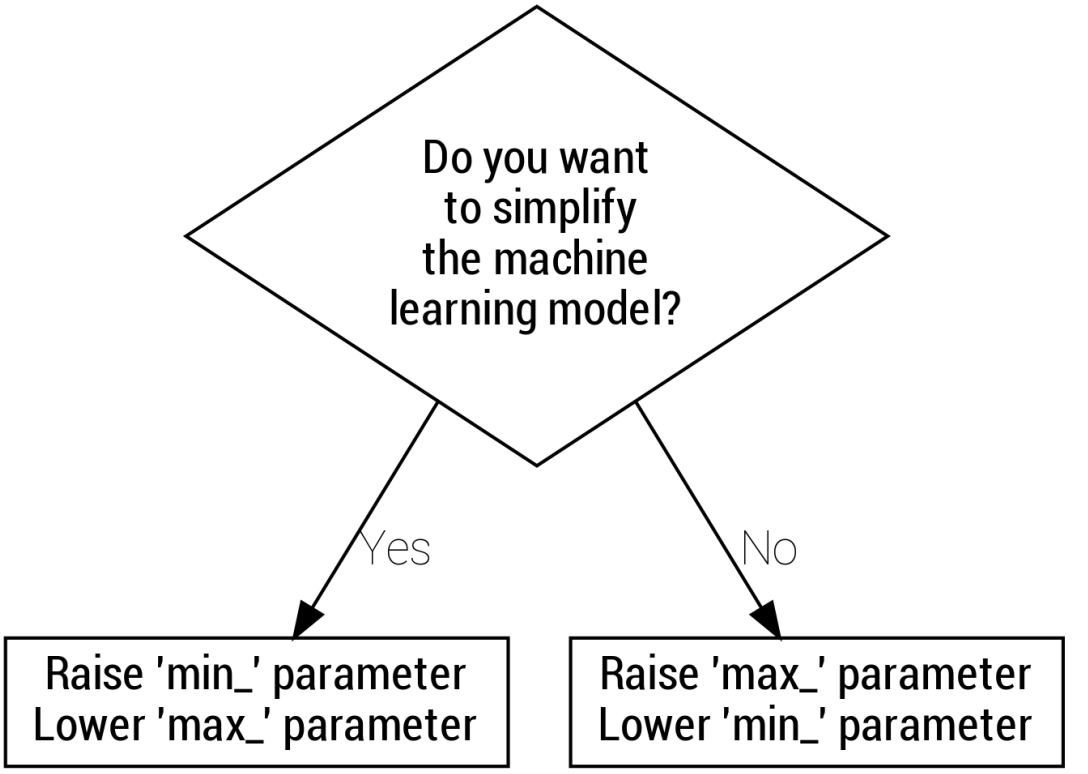


Figure 7.1: Scikit-learn convention for hyperparameter tuning options to deal with overfit and underfit models.

```
>>> stump.get_params()
{'ccp_alpha': 0.0,
 'class_weight': None,
 'criterion': 'gini',
 'max_depth': 1,
 'max_features': None,
 'max_leaf_nodes': None,
 'min_impurity_decrease': 0.0,
 'min_samples_leaf': 1,
 'min_samples_split': 2,
 'min_weight_fraction_leaf': 0.0,
 'random_state': None,
 'splitter': 'best'}
```

We can verify that this is a stump because the `max_depth` hyperparameter is set to 1.

7.2 Tracking changes with Validation Curves

Let's adjust the depth of a tree while tracking accuracy. A chart that does this is called a *Validation Curve*.

With the stump, we get around 62% accuracy. As we add more levels, the accuracy tends to improve until a point. Then it starts to fall back down.

Why does the accuracy start to fall? Our model is now overfitting. Rather than extracting valuable insights from the training data, it memorizes the noise because we allow it to get too complicated. When we try to make a prediction with new data that is missing the same noise, our overfit model underperforms.

Here is the code to create a validation curve:

```
accuracies = []
for depth in range(1, 15):
    between = tree.DecisionTreeClassifier(max_depth=depth)
    between.fit(X_train, kag_y_train)
    accuracies.append(between.score(X_test, kag_y_test))
fig, ax = plt.subplots(figsize=(10,4))
(pd.Series(accuracies, name='Accuracy', index=range(1, len(accuracies)+1))
 .plot(ax=ax, title='Accuracy at a given Tree Depth'))
ax.set_ylabel('Accuracy')
ax.set_xlabel('max_depth')
```

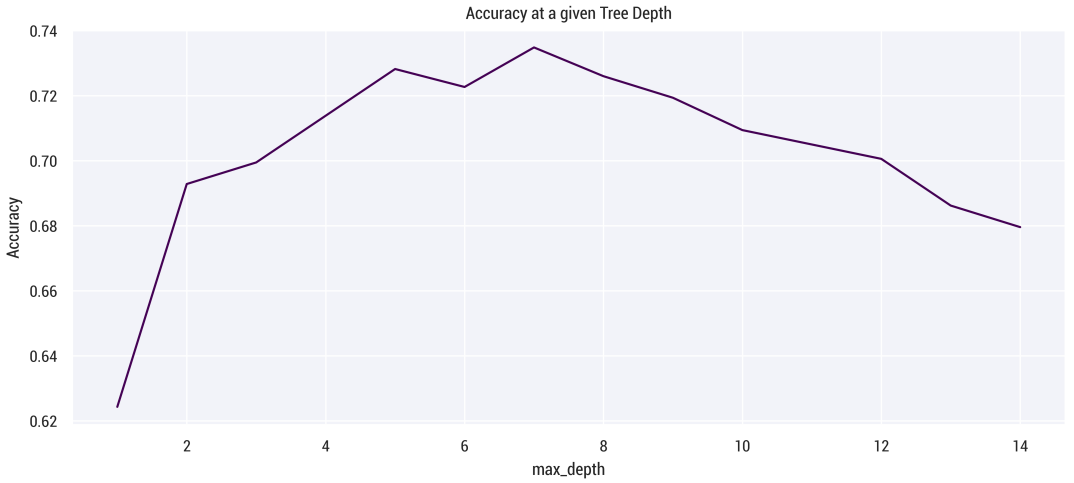


Figure 7.2: A handmade validation curve tracking accuracy over depth.

From this graph, it looks like a depth of seven maximizes the accuracy. Let's check the `.score` of our "Goldilocks" model.

```
>>> between = tree.DecisionTreeClassifier(max_depth=7)
>>> between.fit(X_train, kag_y_train)
>>> between.score(X_test, kag_y_test)
0.7359116022099448
```

Remember that the scores of the underfit and overfit models were 62 and 67%, respectively. An accuracy of 74% is a nice little bump.

7.3 Leveraging Yellowbrick

I come from the lazy programmer school of thought. The code above to track the accuracy while sweeping over hyperparameter changes is not particularly difficult, but I would rather

have one line of code that does that for me. The Yellowbrick library provides a visualizer, `validation_curve`, for us. (Ok, it is a few more lines of code because we need to import it and I'm also configuring Matplotlib and saving the image, but in your notebook, you only need the call to `validation_curve` if you have imported the code.)

```
from yellowbrick.model_selection import validation_curve
fig, ax = plt.subplots(figsize=(10,4))
viz = validation_curve(tree.DecisionTreeClassifier(),
    X=pd.concat([X_train, X_test]),
    y=pd.concat([kag_y_train, kag_y_test]),
    param_name='max_depth', param_range=range(1,14),
    scoring='accuracy', cv=5, ax=ax, n_jobs=6)
```

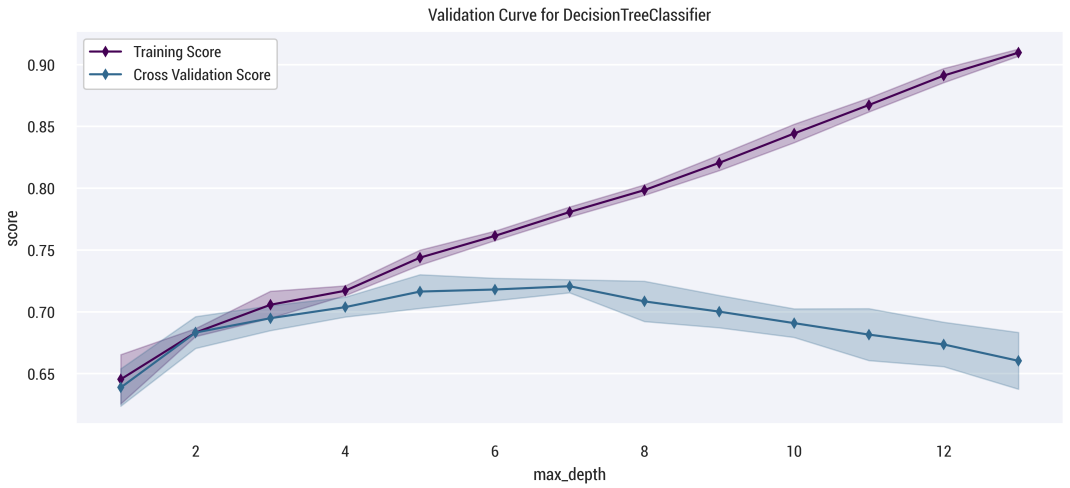


Figure 7.3: Validation curve illustrating and overfitting. Around the depth of 7 appears to be the sweet spot for this data.

This plot is interesting (and better than my hand-rolled one). It has two ones, one tracks the "Training Score", and the other line tracks the "Cross Validation Score". The training score is the accuracy of the training data. You can see that as we increase the tree depth, the accuracy continues to improve. However! We only care about the accuracy of the testing data (this is the "Cross Validation Score"). The testing data line simulates how our model will work when it encounters data it hasn't seen before. The testing data score drops as our model begins to overfit. This plot validates our test and suggests that a depth of seven is an appropriate choice.

The `validation_curve` function also allows us to specify other parameters:

- `cv` - Number of k-fold cross validations (default to 3)
- `scoring` - A scikit-learn model metric string or function (there are various functions in the `sklearn.metrics` module)

The plot also shows a range of scores with the shaded area. This is because it does cross validation and has multiple scores for a given hyperparameter value. Ideally, you want the range to be tight indicating that the metric performed consistently across each split of the cross validation.

7.4 Grid Search

There is a limitation to the validation curve. It only tracks a single hyperparameter. We often want to evaluate many hyperparameters. Grid search is one tool that allows us to experiment across many hyperparameters.

Grid search is one technique used to find the best hyperparameters for a model by training the model on different combinations of hyperparameters and evaluating its performance on a validation set. This technique is often used in decision tree models to tune the hyperparameters that control the tree's growth, such as the maximum depth, minimum samples per leaf, and minimum samples per split.

With grid search, a list of possible values for each hyperparameter is specified, and the model is trained and evaluated on every combination of hyperparameters. For example, if the maximum depth of the decision tree can take values in the range 1 to 10, and the minimum samples per leaf can take values in the range 1 to 100, then a grid search would involve training and evaluating the model on all possible combinations of maximum depth and minimum samples per leaf within these ranges.

After training and evaluating the model on all combinations of hyperparameters, the combination that produces the best performance on the validation set is selected as the optimal set of hyperparameters for the model.

Scikit-learn provides this functionality in the `GridSearchCV` class. We specify a dictionary to map the hyperparameters we want to explore to a list of options. The `GridSearchCV` class follows the standard scikit-learn interface and provides a `.fit` method to kick off the search for the best hyperparameters.

The `verbose=1` parameter is not a hyperparameter, it tells grid search to display computation time for each attempt.

```
from sklearn.model_selection import GridSearchCV
params = {
    'max_depth': [3, 5, 7, 8],
    'min_samples_leaf': [1, 3, 4, 5, 6],
    'min_samples_split': [2, 3, 4, 5, 6],
}
grid_search = GridSearchCV(estimator=tree.DecisionTreeClassifier(),
                           param_grid=params, cv=4, n_jobs=-1,
                           verbose=1, scoring="accuracy")
grid_search.fit(pd.concat([X_train, X_test]),
    pd.concat([kag_y_train, kag_y_test]))
```

After the grid search has exhausted all the combinations, we can inspect the best parameters on the `.best_params_` attribute.

```
>>> grid_search.best_params_
{'max_depth': 7, 'min_samples_leaf': 5, 'min_samples_split': 6}
```

Then I can use these parameters while constructing the decision tree. (The *unpack operation*, **, will use each key from a dictionary as a keyword parameter with its associated value.

```
>>> between2 = tree.DecisionTreeClassifier(**grid_search.best_params_)
>>> between2.fit(X_train, kag_y_train)
>>> between2.score(X_test, kag_y_test)
0.7259668508287292
```

Why is this score less than our *between* tree that we previously created? Generally, this is due to grid search doing *k-fold cross validation*. In our grid search, we split the data into four parts (`cv=4`). Then we train each hyperparameter against three parts and evaluate it against the remaining part. The score is the mean of each of the four testing scores.

When we were only calling the `.score` method, we held out one set of data for evaluation. It is conceivable that the section of data that was held out happens to perform slightly above the average from the grid search.

There is also a `.cv_results_` attribute that contains the scores for each of the options. Here is Pandas code to explore this. The darker cells have better scores.

```
# why is the score different than between_tree?
(pd.DataFrame(grid_search.cv_results_)
 .sort_values(by='rank_test_score')
 .style
 .background_gradient(axis='rows')
)
```

```
: # why is the score different that between_tree?
  (pd.DataFrame(grid_search.cv_results_)
   .sort_values(by='rank_test_score')
   .style
   .background_gradient(axis='rows')
  )
```

:

_samples_split	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	mean_test_score	std_test_score	rank_test_score
5	{'max_depth': 7, 'min_samples_leaf': 5, 'min_samples_split': 5}	0.706897	0.740053	0.712202	0.719788	0.719735	0.012594	1
6	{'max_depth': 7, 'min_samples_leaf': 5, 'min_samples_split': 6}	0.705570	0.737401	0.712202	0.719788	0.718740	0.011890	2
100	{'max_depth': 8, 'min_samples_leaf': 20, 'min_samples_split': 100}	0.698939	0.741379	0.704244	0.729084	0.718411	0.017474	3
100	{'max_depth': 7, 'min_samples_leaf': 20, 'min_samples_split': 100}	0.698939	0.741379	0.704244	0.729084	0.718411	0.017474	3

Figure 7.4: Scores for each split for the hyperparameter values of the grid search.

We can manually validate the results of the grid search using the `cross_val_score`. This scikit-learn function does cross validation on the model and returns the score from each training.

```
>>> results = model_selection.cross_val_score(
...     tree.DecisionTreeClassifier(max_depth=7),
...     X=pd.concat([X_train, X_test], axis='index'),
...     y=pd.concat([kag_y_train, kag_y_test], axis='index'),
...     cv=4
... )
```

```
>>> results
array([0.69628647, 0.73607427, 0.70291777, 0.7184595 ])

>>> results.mean()
0.7134345024851962
```

Here is the same process running against the updated hyperparameters from the grid search. (Note that if you change *cv* to 3, the grid search model performs slightly worse.)

```
>>> results = model_selection.cross_val_score(
...     tree.DecisionTreeClassifier(max_depth=7, min_samples_leaf=5,
...                                 min_samples_split=2),
...     X=pd.concat([X_train, X_test], axis='index'),
...     y=pd.concat([kag_y_train, kag_y_test], axis='index'),
...     cv=4
... )

>>> results
array([0.70822281, 0.73740053, 0.70689655, 0.71580345])

>>> results.mean()
0.7170808366886126
```

One thing you want to be careful of when evaluating machine learning models is that you are comparing apples to apples. Even if you are using the same evaluation metric, you need to make sure that you are evaluating against the same data.

7.5 Summary

In this chapter, we learned that we can modify the accuracy of the model by changing the hyperparameters. We used a validation curve to visualize the impact of a single hyperparameter change and then changed many of them using grid search. Later on, we will see other techniques to tune hyperparameters.

7.6 Exercises

1. What are validation curves and how are they used in hyperparameter tuning?
2. What is grid search and how does it differ from validation curves?
3. What are some pros and cons of validation curves?
4. What are some pros and cons of grid search?

Chapter 8

Random Forest

Before diving into XGBoost, we will look at one more model. The *Random Forest*. This is a common tree-based model that is useful to compare and contrast with XGBoost (indeed, XGBoost can create random forest models if so desired).

8.1 Ensembles with Bagging

Jargon warning! A random forest model is an *ensemble* model. An ensemble is a group of models that aggregate results to prevent overfitting. The ensembling technique used by random forests is called *bagging*. Bagging (which is short for more jargon-*bootstrap aggregating*) means taking the models' average. Bootstrapping means to sample with replacement. In effect, a random forest trains multiple decision trees, but each one is trained on different rows of the data (and different subsets of the features). The prediction is the average of those trees.

Random forests also use *column subsampling*. Column subsampling in a random forest is a technique where at various points in the model or tree creation, a random subset of the features is considered rather than using all the features. This can help to reduce the correlation between the trees and improve the overall performance of the random forest by reducing overfitting and increasing the diversity of the trees.

After the trees have been created, the trees "vote" for the final class.

An intuitive way to think about this comes from a theory proposed in 1785, *Condorcet's jury theorem*. The Marquis de Condorcet was a French philosopher who proposed a mechanism for deciding the correct number of jurors to arrive at the correct verdict. The basic idea is that you should add a juror if they have more than a 50% chance of picking the right verdict and are not correlated with the other jurors. You should keep adding additional jurors if they have $> 50\%$ accuracy. Each juror will contribute more to the correct verdict. Similarly, if a decision tree has a greater than 50% chance of making the right classification (and it is looking at different samples and different features), you should take its vote into consideration. Because we subsample on the columns, that should aid in reducing the correlation between the trees.

8.2 Scikit-learn Random Forest

Here is a random forest model created with scikit-learn:

```
>>> from sklearn import ensemble
>>> rf = ensemble.RandomForestClassifier(random_state=42)
>>> rf.fit(X_train, kag_y_train)
>>> rf.score(X_test, kag_y_test)
0.7237569060773481
```

How many trees did this create? Let's look at the hyperparameters:

```
>>> rf.get_params()
{'bootstrap': True,
 'ccp_alpha': 0.0,
 'class_weight': None,
 'criterion': 'gini',
 'max_depth': None,
 'max_features': 'sqrt',
 'max_leaf_nodes': None,
 'max_samples': None,
 'min_impurity_decrease': 0.0,
 'min_samples_leaf': 1,
 'min_samples_split': 2,
 'min_weight_fraction_leaf': 0.0,
 'n_estimators': 100,
 'n_jobs': None,
 'oob_score': False,
 'random_state': 42,
 'verbose': 0,
 'warm_start': False}
```

It looks like it made 100 trees (see *n_estimators*). And if we want to, we can explore each tree. They are found under the `.estimators_` attribute.

Note

Scikit-learn has many conventions. We discussed how raising *max_* hyperparameters would make the model more complex.

Another convention is that any attribute that ends with an underscore (_), like `.estimators_`, is created during the call to the `.fit` method. Sometimes these will be insights, metadata, or tables of data.

In this case, it is a list of the trees created while training the model.

We can validate that there are 100 trees.

```
>>> len(rf.estimators_)
100
```

Every "estimator" is a decision tree. Here is the first tree.

```
>>> print(rf.estimators_[0])
DecisionTreeClassifier(max_features='sqrt', random_state=1608637542)
```

We can visualize each tree if desired. Here is the visualization for tree 0 (limited to depth two so we can read it).

```
fig, ax = plt.subplots(figsize=(8, 4))
features = list(c for c in X_train.columns)
tree.plot_tree(rf.estimators_[0], feature_names=features,
               filled=True, class_names=rf.classes_, ax=ax,
               max_depth=2, fontsize=6)
```

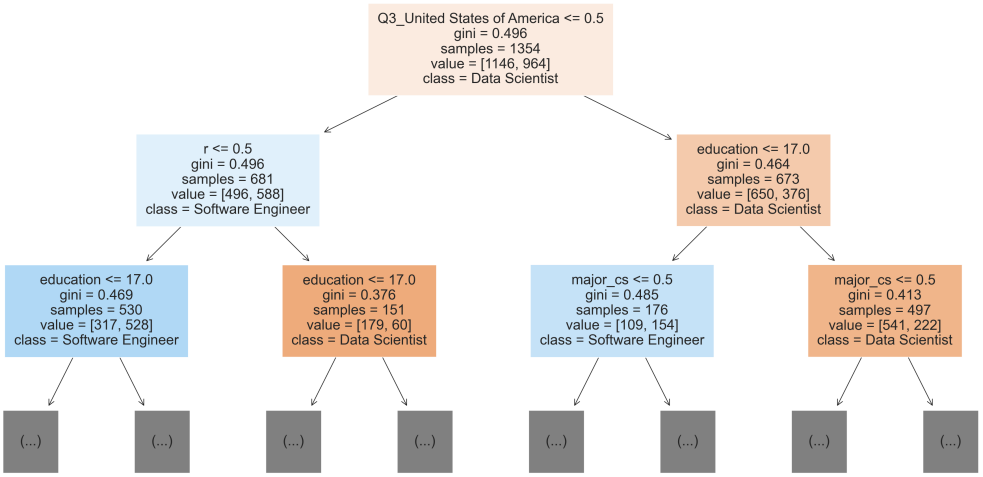


Figure 8.1: A visualization of the first two levels of tree 0 from our random forest.

Interestingly, this tree chose *Q3_United States of America* as the first column. It did not choose the *R* column, probably due to column subsampling.

8.3 XGBoost Random Forest

You can also create a random forest with the XGBoost library.

```
>>> import xgboost as xgb
>>> rf_xg = xgb.XGBRFClassifier(random_state=42)
>>> rf_xg.fit(X_train, y_train)
>>> rf_xg.score(X_test, y_test)
0.7447513812154696
```

You can also inspect the hyperparameters of this random forest. XGBoost uses different hyperparameters than scikit-learn.

```
>>> rf_xg.get_params()
{'colsample_bynode': 0.8,
 'learning_rate': 1.0,
 'reg_lambda': 1e-05,
 'subsample': 0.8,
 'objective': 'binary:logistic',
 'use_label_encoder': None,
 'base_score': 0.5,
 'booster': 'gbtree',
 'callbacks': None,
 'colsample_bylevel': 1,
 'colsample_bytree': 1,
 'early_stopping_rounds': None,
 'enable_categorical': False,
```

```
 'eval_metric': None,
 'feature_types': None,
 'gamma': 0,
 'gpu_id': -1,
 'grow_policy': 'depthwise',
 'importance_type': None,
 'interaction_constraints': '',
 'max_bin': 256,
 'max_cat_threshold': 64,
 'max_cat_to_onehot': 4,
 'max_delta_step': 0,
 'max_depth': 6,
 'max_leaves': 0,
 'min_child_weight': 1,
 'missing': nan,
 'monotone_constraints': '()',
 'n_estimators': 100,
 'n_jobs': 0,
 'num_parallel_tree': 100,
 'predictor': 'auto',
 'random_state': 42,
 'reg_alpha': 0,
 'sampling_method': 'uniform',
 'scale_pos_weight': 1,
 'tree_method': 'exact',
 'validate_parameters': 1,
 'verbosity': None}
```

Let's visualize the first tree with the `plot_tree` function of XGBoost.

```
fig, ax = plt.subplots(figsize=(6,12), dpi=600)
xgb.plot_tree(rf_xg, num_trees=0, ax=ax, size='1,1')
```

```
my_dot_export(rf_xg, num_trees=0, filename='img/rf_xg_kag.dot',
              title='First Random Forest Tree', direction='LR')
```

That tree is big, and a little hard to see what is going on without a microscope. Let's use the `dtreeviz` library and limit the depth to 2 (`depth_range_to_display=[0,2]`).

```
viz = dtreeviz.model(rf_xg, X_train=X_train,
    y_train=y_train,
    target_name='Job', feature_names=list(X_train.columns),
    class_names=['DS', 'SE'], tree_index=0)
viz.view(depth_range_to_display=[0,2])
```

8.4 Random Forest Hyperparameters

Here are most of the hyperparameters for random forests (created with the XGBoost library). At a high level, you have the tree parameters (like the depth), sampling parameters, the ensembling parameters (number of trees), and regularization parameters (like gamma, which prunes back trees to regularize them).

Tree hyperparameters.

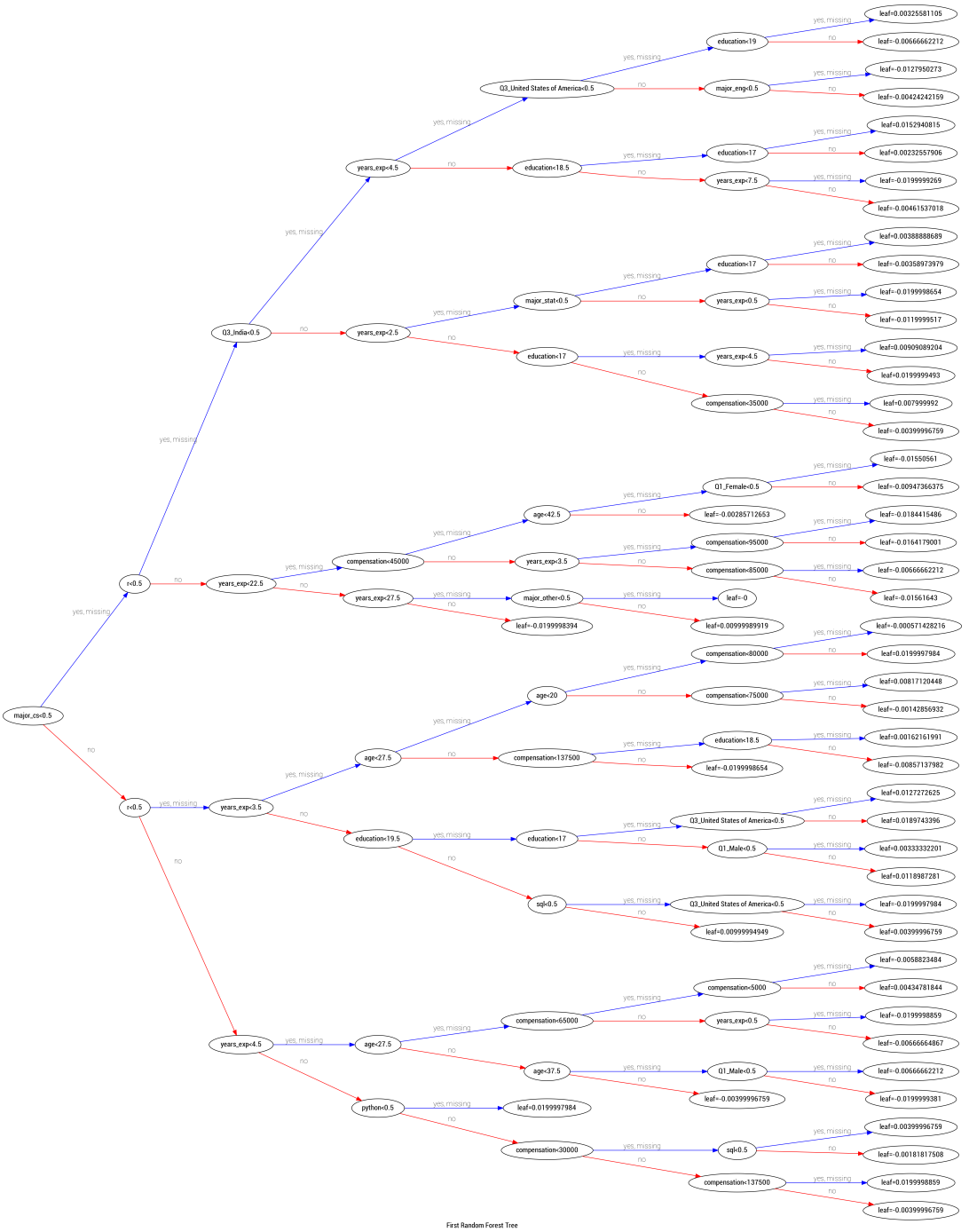


Figure 8.2: First random forest tree built with XGBoost.

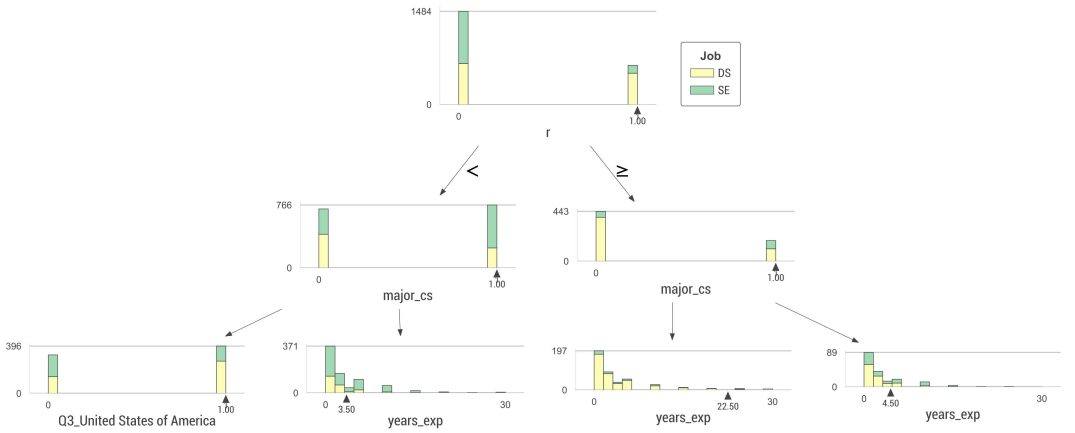


Figure 8.3: dtreeviz export of XGBoost Random Forest model.

- `max_depth=6` - Tree depth. Range [1-large number]
- `max_leaves=0` - Number of leaves in a tree. (Not supported by `tree_method='exact'`). A value of 0 means no limit, otherwise, the range is [2-large number]
- `min_child_weight=1` - Minimum sum of hessian (weight) needed in a child. The larger the value, the more conservative. Range [0,∞]
- `grow_policy="depthwise"` - (Only supported with `tree_method` set to `'hist'`, `'approx'`, or `'gpu_hist'`. Split nodes closest to root. Set to `"lossguide"` (with `max_depth=0`) to mimic LightGBM behavior.
- `tree_method="auto"` - Use `"hist"` to use histogram bucketing and increase performance. `"auto"` heuristic:
  - `'exact'` for small data
  - `'approx'` for larger data
  - `'hist'` for histograms (limit bins with `max_bins` (default 256))

Sampling hyperparameters. These work cumulatively. First by the tree, then by level, and finally by the node.

- `colsample_bytree=1` - Subsample columns at each tree. Range (0,1]
- `colsample_bylevel=1` - Subsample columns at each level (from tree columns `colsample_bytree`). Range (0,1]
- `colsample_bynode=1` - Subsample columns at each node split (from tree columns `colsample_bytree`). Range (0,1]
- `subsample=1` - Sample portion of training data (rows) before each boosting round. Range (0,1]
- `sampling_method='uniform'` - Sampling method. `'uniform'` for equal probability. `gradient_based` sample is proportional to the regularized absolute value of gradients (only supported by `tree_method="gpu_hist"`.

Categorical data hyperparameters. Use the `enable_categorical=True` parameter and set columns to Pandas categoricals (`.astype('category')`).

- `max_cat_to_onehot=4` - Upper threshold for using one-hot encoding for categorical features. Use one-hot encoding when the number of categories is less than this number.

- `max_cat_threshold=64` - Maximum number of categories to consider for each split.

Ensembling hyperparameters.

- `n_estimators=100` - Number of trees. Range [1, large number]
- `early_stopping_rounds=None` - Stop creating new trees if `eval_metric` score has not improved after n rounds. Range [1,∞]
- `eval_metric` - Metric for evaluating validation data.
  - Classification metrics
    - `'logloss'` - Default classification metric. Negative log-likelihood.
    - `'auc'` - Area under the curve of Receiver Operating Characteristic.
- `objective` - Learning objective to optimize the fit of the model to data during training.
  - Classification objectives
    - `'binary:logistic'` - Default classification objective. Outputs probability.

Regularization hyperparameters.

- `learning_rate=.3` - After each boosting round, multiply weights to make it more conservative. Lower is more conservative. Range (0,1]
- `gamma=0` - Minimum loss required to make a further partition. Larger is more conservative. Range [0,∞]
- `reg_alpha=0` - L1 regularization. Increasing will make it more conservative.
- `reg_lambda=1` - L2 regularization. Increasing will make it more conservative.

Imbalanced data hyperparameters.

- `scale_pos_weight=1` - Consider (count negative) / (count positive) for imbalanced classes. Range (0, large number)
- `max_delta_step=0` - Maximum delta step for leaf output. It might help with imbalanced classes. Range [0,∞]

8.5 Training the Number of Trees in the Forest

Let's try out a few different values for `n_estimators` and inspect what happens to the accuracy of our random forest model.

```
from yellowbrick.model_selection import validation_curve
fig, ax = plt.subplots(figsize=(10,4))
viz = validation_curve(xgb.XGBClassifier(random_state=42),
    x=pd.concat([X_train, X_test], axis='index'),
    y=np.concatenate([y_train, y_test]),
    param_name='n_estimators', param_range=range(1, 100, 2),
    scoring='accuracy', cv=3,
    ax=ax)
```

It looks like our model performs best around 29 estimators. A quick check shows that it performs slightly better than the out-of-the-box model. I will choose the simpler model when given a choice between models that perform similarly. In this case, a model with 99 trees appears to perform similarly but is more complex, so I go with the simpler model.

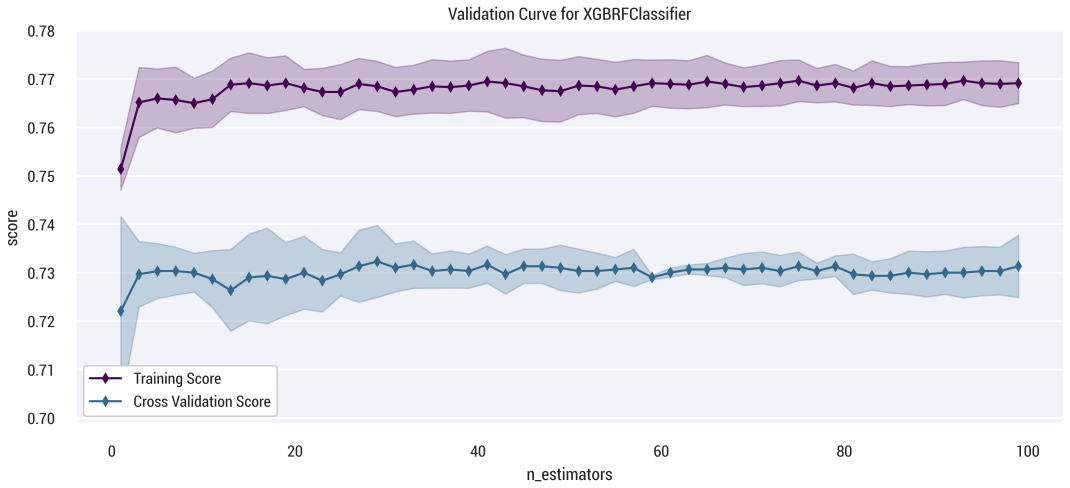


Figure 8.4: Validation curve for random forest model that shows accuracy score as n_estimators changes. Prefer the simpler model when two models return similar results.

```
>>> rf_xg29 = xgb.XGBRFClassifier(random_state=42, n_estimators=29)
>>> rf_xg29.fit(X_train, y_train)
>>> rf_xg29.score(X_test, y_test)
0.7480662983425415
```

8.6 Summary

In this chapter, we introduced a random forest. At its essence, a random forest is a group of decision trees formed by allowing each tree to sample different rows of data. Each tree can look at different features as well. The hope is that the trees can glean different insights because they look at different parts of the data. Finally, the results of the trees are combined to predict a final result.

8.7 Exercises

1. How do we specify the hyperparameters for an XGBoost model to create a random forest?
2. Create a random forest for the Kaggle data predicting whether the title is a data scientist or software engineer.
3. What is the score?
4. You have two models with the same score. One has 200 trees; the other has 40. Which model do you choose and why?

Chapter 9

XGBoost

At last, we can talk about the most common use of the XGBoost library, making extreme gradient-boosted models.

This ensemble is created by training a *weak model* and then training another model to correct the *residuals* or mistakes of the model before it.

I like to use a golfing metaphor. A decision tree is like getting one swing to hit the ball and put it in the hole. A random forest is getting a bunch of different attempts at teeing off (changing clubs, swing styles, etc.) and then placing the ball at the average of each of those swings. A boosted tree is hitting the ball once, then going to where it landed and hitting again (trying to correct the mistake of the previous hit), and then getting to keep hitting the ball. This metaphor clarifies why boosted models work so well; they can correct the model's error as they progress.

9.1 Jargon

I used the term *weak model* to describe a single tree in boosting. A weak tree is a shallow tree. Boosting works well for performance reasons. Deeper trees grow nodes at a fast rate. Using many shallower trees tends to provide better performance and quicker results. However, shallow trees might not be able to capture interactions between features.

I also used the term *residual*. This is the prediction error, the true value minus the predicted value. For a regression model, where we predict a numeric value, this is the difference between the predicted and actual values. If the residual is positive, it means our prediction was too high. A negative residual means that we underpredicted. A residual of zero means that the prediction was correct.

For classification tasks, where we are trying to predict a label, you can think of the residual in terms of probabilities. For a positive label, we would prefer a probability greater than .5. A perfect model would have a probability of 1. If our model predicted a probability of .4, the residual is .6, the perfect value (1) minus the predicted value (.4). The subsequent boosting round would try to predict .6 to correct that value.

9.2 Benefits of Boosting

The XBGoost library provides extras we don't get with a standard decision tree. It has support to handle missing values. It also has support for learning from categorical features. Additionally, it has various hyperparameters to tune model behavior and fend off overfitting.

9.3 A Big Downside

However, using XGBoost versus a plain decision tree has a pretty big downside. A decision tree is easy to understand. I've been to a doctor's appointment where I was diagnosed using a decision tree (the doctor showed me the choices after). Understanding how a model works is called *explainability* or *interpretability* and is valuable in some business contexts.

Explaining the reasoning for rejecting the loan might help customers feel like they understand the process. For example, if you were applying for a loan and the bank rejected you, you might be disappointed. But, if the bank told you that if you deposit $2,000 into your account, you would be approved for the loan, you might resolve to save that amount and apply again in the future.

Now, imagine that a data scientist at the bank claims they have a new model better at predicting who will default on a loan, but this model cannot explain why. The same user comes in and applies for the loan. They are rejected again but not given a reason. That could be a big turn-off and cause the customer to look for a new bank. (Perhaps not particularly problematic for folks with a high probability of defaulting on the loan.) However, if enough customers get turned off and start looking to move their business elsewhere, that could hurt the bottom line.

In short, many businesses are willing to accept models with worse performance but are interpretable. Models that are easy to explain are called *white box models*. Conversely, hard-to-explain models are called *black box models*.

To be pedantic, you can explain a boosted model. However, if there are 500 trees in it, that might be a frustrating experience to walk a customer through each of those trees in order.

9.4 Creating an XGBoost Model

Let's load the libraries and the `xg_helpers` library. The `xg_helpers` library has the Python functions I created to load the data and prepare it for modeling.

```
%matplotlib inline

import dtreeviz
from feature_engine import encoding, imputation
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn import base, compose, datasets, ensemble, \
    metrics, model_selection, pipeline, preprocessing, tree
import scikitplot
import xgboost as xgb
import yellowbrick.model_selection as ms
from yellowbrick import classifier

import urllib
import zipfile

import xg_helpers as xhelp

url = 'https://github.com/mattharrison/datasets/raw/master/data/'\
    'kaggle-survey-2018.zip'
fname = 'kaggle-survey-2018.zip'
member_name = 'multipleChoiceResponses.csv'
```

```
raw = xhelp.extract_zip(url, fname, member_name)


## Create raw X and raw y
kag_X, kag_y = xhelp.get_rawX_y(raw, 'Q6')

## Split data
kag_X_train, kag_X_test, kag_y_train, kag_y_test = \
    model_selection.train_test_split(
        kag_X, kag_y, test_size=.3, random_state=42, stratify=kag_y)

## Transform X with pipeline
X_train = xhelp.kag_pl.fit_transform(kag_X_train)
X_test = xhelp.kag_pl.transform(kag_X_test)

## Transform y with label encoder
label_encoder = preprocessing.LabelEncoder()
label_encoder.fit(kag_y_train)
y_train = label_encoder.transform(kag_y_train)
y_test = label_encoder.transform(kag_y_test)

# Combined Data for cross validation/etc
X = pd.concat([X_train, X_test], axis='index')
y = pd.Series([*y_train, *y_test], index=X.index)
```

Now we have our training data and test data and are ready to create models.

9.5 A Boosted Model

The XGBoost library makes it really easy to convert from a decision tree to a random forest or a boosted model. (The scikit-learn library also makes trying other models, like logistic regression, support vector machines, or k-nearest neighbors easy.) Let's make our first boosted model.

All we need to do is create an instance of the model (generally, this is the only thing we need to change to try a different model). Then we call the `.fit` method to train the model on the training data.

We can evaluate the model using the `.score` method, which tells us the accuracy of the model. Out of the box, the accuracy of the boosted model (.745) is quite a bit better than both a decision tree (.73 for a depth of 7) and a random forest (.72 from scikit-learn).

```
>>> xg_oob = xgb.XGBClassifier()
>>> xg_oob.fit(X_train, y_train)
>>> xg_oob.score(X_test, y_test)
0.7458563535911602
```

The default model performs 100 rounds of boosting. (Again, this is like getting 100 chances to keep hitting the golf ball and move it toward the hole.) Let's see what happens to our accuracy if we only hit two times (limiting the tree estimators to two and the depth of each tree to two).

```
>>> # Let's try w/ depth of 2 and 2 trees
>>> xg2 = xgb.XGBClassifier(max_depth=2, n_estimators=2)
>>> xg2.fit(X_train, y_train)
>>> xg2.score(X_test, y_test)
0.6685082872928176
```

Our performance dropped quite a bit. Down to .668. Playing around with these hyperparameters can have a large impact on our model's performance.

Let's look at what the first tree looks like (We will use `tree_index=0` to indicate this). I will use the `dtreeviz` package to show the tree. It looks like it first considers whether the user uses the R programming language. Then it looks at whether the user studied CS. (Note that it doesn't have to look at the same feature for a level, but in this case, the CS feature turns out to be most informative for both splits.)

```
import dtreeviz

viz = dtreeviz.model(xg2, X_train=X, y_train=y, target_name='Job',
    feature_names=list(X_train.columns),
    class_names=['DS', 'SE'], tree_index=0)
viz.view(depth_range_to_display=[0,2])
```

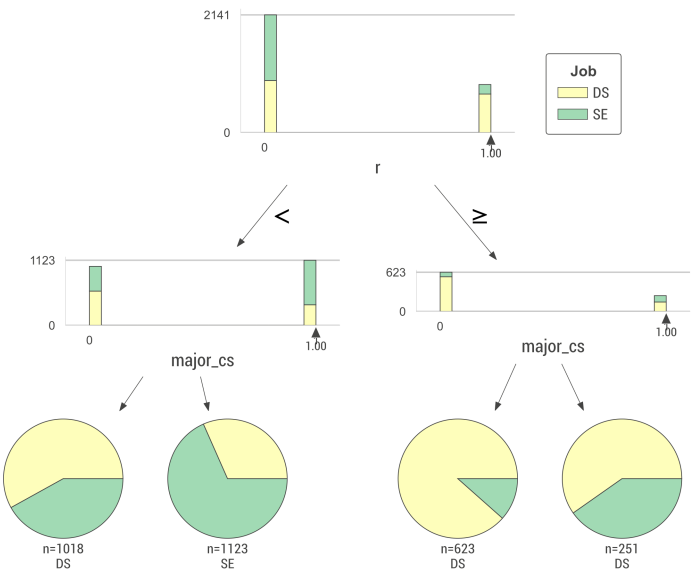


Figure 9.1: Visualization of first booster from xgboost model.

The `dtreeviz` package is good for understanding what the trees are doing and how the splits break up the groups.

9.6 Understanding the Output of the Trees

One thing that the `dtreeviz` does not show with XGBoost models is the score for the nodes.

Let's use XGBoost to plot the same booster (using `num_trees=0`) and trace through an example to understand the leaf scores.

I'm using my function, `my_dot_export`, because it uses my fonts.

```
xhelp.my_dot_export(xg2, num_trees=0, filename='img/xgb_md2.dot',
                    title='First Tree')
```

Note

You can just use the `plot_tree` function if you want to view this in a Jupyter notebook

```
xgb.plot_tree(xg2, num_trees=0)
```

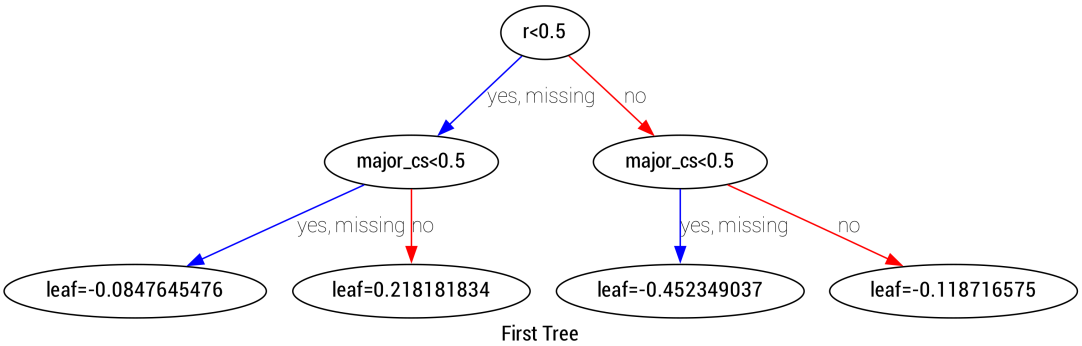


Figure 9.2: Visualization of first tree from xgboost model.

Let's trace through an example and see what our model predicts.

Here is the prediction for this two-tree model. It predicts 49.8% data scientist and 50.1% software engineer. Because 50.1% is greater than 50%, our model predicts 1 or software engineer for this example.

We can ask the model to predict probabilities with `.predict_proba`.

```
>>> # Predicts 1 - Software engineer
>>> se7894 = pd.DataFrame({'age': {7894: 22},
... 'education': {7894: 16.0},
... 'years_exp': {7894: 1.0},
... 'compensation': {7894: 0},
... 'python': {7894: 1},
... 'r': {7894: 0},
... 'sql': {7894: 0},
... 'Q1_Male': {7894: 1},
... 'Q1_Female': {7894: 0},
... 'Q1_Prefer not to say': {7894: 0},
... 'Q1_Prefer to self-describe': {7894: 0},
... 'Q3_United States of America': {7894: 0},
... 'Q3_India': {7894: 1},
... 'Q3_China': {7894: 0},
... 'major_cs': {7894: 0},
... 'major_other': {7894: 0},
... 'major_eng': {7894: 0},
... 'major_stat': {7894: 0}})
>>> xg2.predict_proba(se7894)
array([[0.4986236, 0.5013764]], dtype=float32)
```

Or we can predict just the value with `.predict`:

```
>>> # Predicts 1 - Software engineer
>>> xg2.predict(pd.DataFrame(se7894))
array([1])
```

For our example, the user did not use R, so we took a left turn at the first node. They did not study CS, so we moved left at that node and ended with a leaf value of -0.084.

Now let's trace what happens in the second tree.

First, let's plot it. Remember that this tree tries to fix the predictions of the tree before it. (We specify `num_trees=1` for the second tree because Python is zero-based.)

```
xhelp.my_dot_export(xg2, num_trees=1, filename='img/xgb_md2_tree1.dot', title='Second Tree')
```

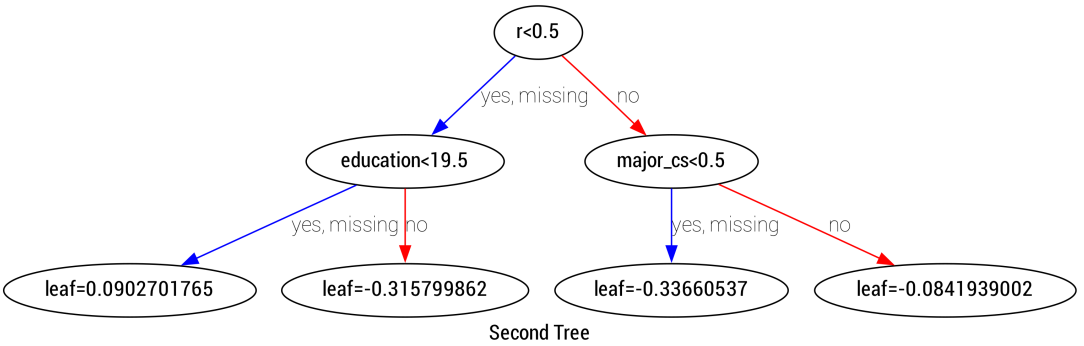


Figure 9.3: Visualization of second booster from xgboost model.

Again, our example does not use R, so we go left on the first node. The education level is 16, so we take another left at the next node and end at a value of 0.0902.

Our prediction is made by adding these leaf values together and taking the inverse logit of the sum. This returns the probability of the positive class.

```
def inv_logit(p):
    return np.exp(p) / (1 + np.exp(p))

>>> inv_logit(-0.08476+0.0902701)
0.5013775215147345
```

We calculate the inverse logit of the sum of the leaf values and come up with .501. Again this corresponds to 50.1% software engineer (with binary classifiers, this is the percent that the label is 1, or software engineer).

If there were more trees, we would repeat this process and add up the values in the leaves. Note that due to the inverse logistic function, values above 0 push towards the positive label, and values below zero push to the negative label.

9.7 Summary

XGBoost can improve the performance of traditional decision trees and random forest techniques. Decision trees are simpler models and tend to underfit in certain situations, while random forests are better at generalizing to unseen data points. XGBoost combines the benefits of both approaches by using decision trees as its base learners and building an ensemble of trees with the help of boosting. It works by training multiple decision trees with different parameters and combining the results into a powerful model. XGBoost also has additional features such as regularization, early stopping, and feature importance making it more effective than regular decision tree and random forest algorithms.

9.8 Exercises

1. What are the benefits of an XGBoost model versus a Decision Tree model?
2. Create an XGBoost model with a dataset of your choosing for classification. What is the accuracy score?
3. Compare your model with a Decision Tree model.
4. Visualize the first tree of your model.

Chapter 10

Early Stopping

Early stopping in XGBoost is a technique that reduces overfitting in XGBoost models. You can tell XGBoost to use early stopping when you call the `.fit` method. Early stopping works by monitoring the performance of a model on a validation set and automatically stopping training when the model's performance on the validation set no longer improves. This helps to improve the model's generalization, as the model is not overfitted to the training data. Early stopping is helpful because it can prevent overfitting and help to improve XGBoost models' generalization.

10.1 Early Stopping Rounds

Here are the results of XGBoost using out-of-the-box behavior. By default, the model will create 100 trees. This model labels about 74% of the test examples correctly.

```
>>> # Defaults
>>> xg = xgb.XGBClassifier()
>>> xg.fit(X_train, y_train)
>>> xg.score(X_test, y_test)
0.7458563535911602
```

Now, we are going to provide the `early_stopping_rounds` parameter. Note that we also specify an `eval_set` parameter. The `eval_set` parameter is used in the `.fit` method to provide a list of (X, y) tuples for evaluation to use as a validation dataset. This dataset evaluates the model's performance and adjusts the hyperparameters during the model training process. We pass in two tuples, the training data and the testing data. When you combine the evaluation data with the early stopping parameter, XGBoost will look at the results of the last tuple. We passed in 20 for `early_stopping_rounds`, so XGBoost will build up to 100 trees, but if the log loss hasn't improved after the following 20 trees, it will stop building trees.

You can see the line where the log loss bottoms out. Note that the score on the left, 0.43736, is the training score and the score on the right, 0.5003, is the testing score and the value used for early stopping.

```
[12]    validation_0-logloss:0.43736    validation_1-logloss:0.5003
```

(Note that this is the thirteenth tree because we started counting at 0.)

```
>>> xg = xgb.XGBClassifier(early_stopping_rounds=20)
>>> xg.fit(X_train, y_train,
...        eval_set=[(X_train, y_train),
...                  (X_test, y_test)
```

```
...                ]
...       )
>>> xg.score(X_test, y_test)
[0] validation_0-logloss:0.61534    validation_1-logloss:0.61775
[1] validation_0-logloss:0.57046    validation_1-logloss:0.57623
[2] validation_0-logloss:0.54011    validation_1-logloss:0.55333
[3] validation_0-logloss:0.51965    validation_1-logloss:0.53711
[4] validation_0-logloss:0.50419    validation_1-logloss:0.52511
[5] validation_0-logloss:0.49176    validation_1-logloss:0.51741
[6] validation_0-logloss:0.48159    validation_1-logloss:0.51277
[7] validation_0-logloss:0.47221    validation_1-logloss:0.51040
[8] validation_0-logloss:0.46221    validation_1-logloss:0.50713
[9] validation_0-logloss:0.45700    validation_1-logloss:0.50583
[10]    validation_0-logloss:0.45062    validation_1-logloss:0.50430
[11]    validation_0-logloss:0.44533    validation_1-logloss:0.50338
[12]    validation_0-logloss:0.43736    validation_1-logloss:0.50033
[13]    validation_0-logloss:0.43399    validation_1-logloss:0.50034
[14]    validation_0-logloss:0.43004    validation_1-logloss:0.50192
[15]    validation_0-logloss:0.42550    validation_1-logloss:0.50268
[16]    validation_0-logloss:0.42169    validation_1-logloss:0.50196
[17]    validation_0-logloss:0.41854    validation_1-logloss:0.50223
[18]    validation_0-logloss:0.41485    validation_1-logloss:0.50360
[19]    validation_0-logloss:0.41228    validation_1-logloss:0.50527
[20]    validation_0-logloss:0.40872    validation_1-logloss:0.50839
[21]    validation_0-logloss:0.40490    validation_1-logloss:0.50623
[22]    validation_0-logloss:0.40280    validation_1-logloss:0.50806
[23]    validation_0-logloss:0.39942    validation_1-logloss:0.51007
[24]    validation_0-logloss:0.39807    validation_1-logloss:0.50987
[25]    validation_0-logloss:0.39473    validation_1-logloss:0.51189
[26]    validation_0-logloss:0.39389    validation_1-logloss:0.51170
[27]    validation_0-logloss:0.39040    validation_1-logloss:0.51218
[28]    validation_0-logloss:0.38837    validation_1-logloss:0.51135
[29]    validation_0-logloss:0.38569    validation_1-logloss:0.51202
[30]    validation_0-logloss:0.37945    validation_1-logloss:0.51352
[31]    validation_0-logloss:0.37840    validation_1-logloss:0.51545
0.7558011049723757
```

We can ask the model what the limit was by inspecting the `.best_ntree_limit` attribute:

```
>>> xg.best_ntree_limit
13
```

Note

If this were implemented in scikit-learn, the attribute `.best_ntree_limit` would have a trailing underscore because it was learned by fitting the model. Alas, we live in a world of inconsistencies.

10.2 Plotting Tree Performance

Let's explore the score that occurred during fitting the model. The `.eval_results` method will return a data structure containing the results from the `eval_set`.

```
>>> # validation_0 is for training data
>>> # validation_1 is for testing data
>>> results = xg.evals_result()
>>> results
{'validation_0': OrderedDict([('logloss',
               [0.6153406503923696,
                0.5704566627034644,
                0.5401074953836288,
                0.519646179894983,
                0.5041859194071372,
                0.49175883369140716,
                0.4815858465553177,
                0.4722135672319274,
                0.46221246084118905,
                0.4570046103131291,
                0.45062119092139025,
                0.44533101600634545,
                0.4373589513231934,
                0.4339914069003403,
                0.4300442738158372,
                0.42550266018419824,
                0.42168949383456633,
                0.41853931894949614,
                0.41485192559138645,
                0.4122836278413833,
                0.4087179538231096,
                0.404898268053467,
                0.4027963532207719,
                0.39941699938733854,
                0.3980718078477953,
                0.39473153180519993,
                0.39388538948800944,
                0.39039599470886893,
                0.38837148147752126,
                0.38569152626668,
                0.3794510693344513,
                0.37840359436957194,
                0.37538466192241227])]),
 'validation_1': OrderedDict([('logloss',
               [0.6177459120091813,
                0.5762297115602546,
                0.5533292921537852,
                0.5371078260695736,
                0.5251118483299708,
                0.5174100387491574,
                0.5127666981510036,
                0.5103968678752362,
                0.5071349115538004,
                0.5058257413585542,
                0.5043005662687247,
                0.5033770955193438,
                0.5003349146419797,
```

```
                0.5003436393562437,
                0.5019165392779843,
                0.502677517614806,
                0.501961292550791,
                0.5022262006329157,
                0.5035970173261607,
                0.5052709663297096,
                0.508388655664636,
                0.5062287504923689,
                0.5080608455824424,
                0.5100736726054829,
                0.5098673969229365,
                0.5118910041889845,
                0.5117007332982608,
                0.5121825202836434,
                0.5113475993625531,
                0.5120185821281118,
                0.5135189292720874,
                0.5154504034915188,
                0.5158137131755071])]}
```

Let's plot that data and visualize what happens as we add more trees:

```
# Testing score is best at 13 trees
results = xg.evals_result()
fig, ax = plt.subplots(figsize=(8, 4))
ax = (pd.DataFrame({'training': results['validation_0']['logloss'],
                    'testing': results['validation_1']['logloss']})
  .assign(ntrees=lambda adf: range(1, len(adf)+1))
  .set_index('ntrees')
  .plot(figsize=(5,4), ax=ax,
        title='eval_results with early_stopping')
)
ax.annotate('Best number \nof trees (13)', xy=(13, .498),
           xytext=(20,.42), arrowprops={'color':'k'})
ax.set_xlabel('ntrees')
```

Now, let's train a model with 13 trees (also referred to as estimators) and see how the model performs:

```
# Using value from early stopping gives same result
>>> xg13 = xgb.XGBClassifier(n_estimators=13)
>>> xg13.fit(X_train, y_train,
...          eval_set=[(X_train, y_train),
...                    (X_test, y_test)]
... )
>>> xg13.score(X_test, y_test)
```

It looks like this model gives the same results that the `early_stopping` model did.

```
>>> xg.score(X_test, y_test)
0.7558011049723757
```

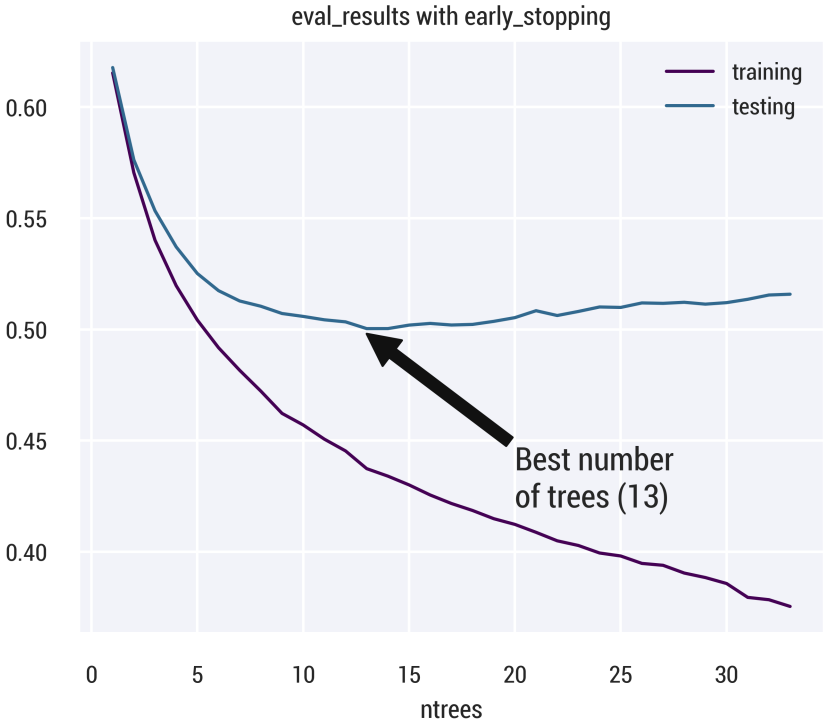


Figure 10.1: Image showing that the testing score might stop improving

Finally, let's look at the model that creates exactly 100 trees. The score is worse than only using 13!

```
>>> # No early stopping, uses all estimators
>>> xg_no_es = xgb.XGBClassifier()
>>> xg_no_es.fit(X_train, y_train)
>>> xg_no_es.score(X_test, y_test)
0.7458563535911602
```

What is the takeaway from this chapter? The `early_stopping_rounds` parameter helps to prevent overfitting.

10.3 Different `eval_metrics`

The `eval_metric` options for XGBoost are metrics used to evaluate the results of an XGBoost model. They provide a way to measure the performance of a model on a dataset. Some of the most common `eval_metric` options for classification are:

1. Log loss (`'logloss'`) - The default measure. Log loss measures a model's negative log-likelihood performance that predicts a given class's probability. It penalizes incorrect predictions more heavily than correct ones.

2. Area under the curve (`'auc'`) - AUC is used to measure the performance of a binary classification model. It is the Area Under a receiver operating characteristic Curve.